

Concurrency (1)

LTS, composition, linear time properties

FDD3024 2026

Roberto Guanciale

Where do we have concurrency?

Hardware: Different parts of the circuit operating at the same time

Processes: interacting processes on an OS that interact via system calls

SW/HW: Device drivers interacting with peripherals

Distributed systems: Different agents communicating with each others

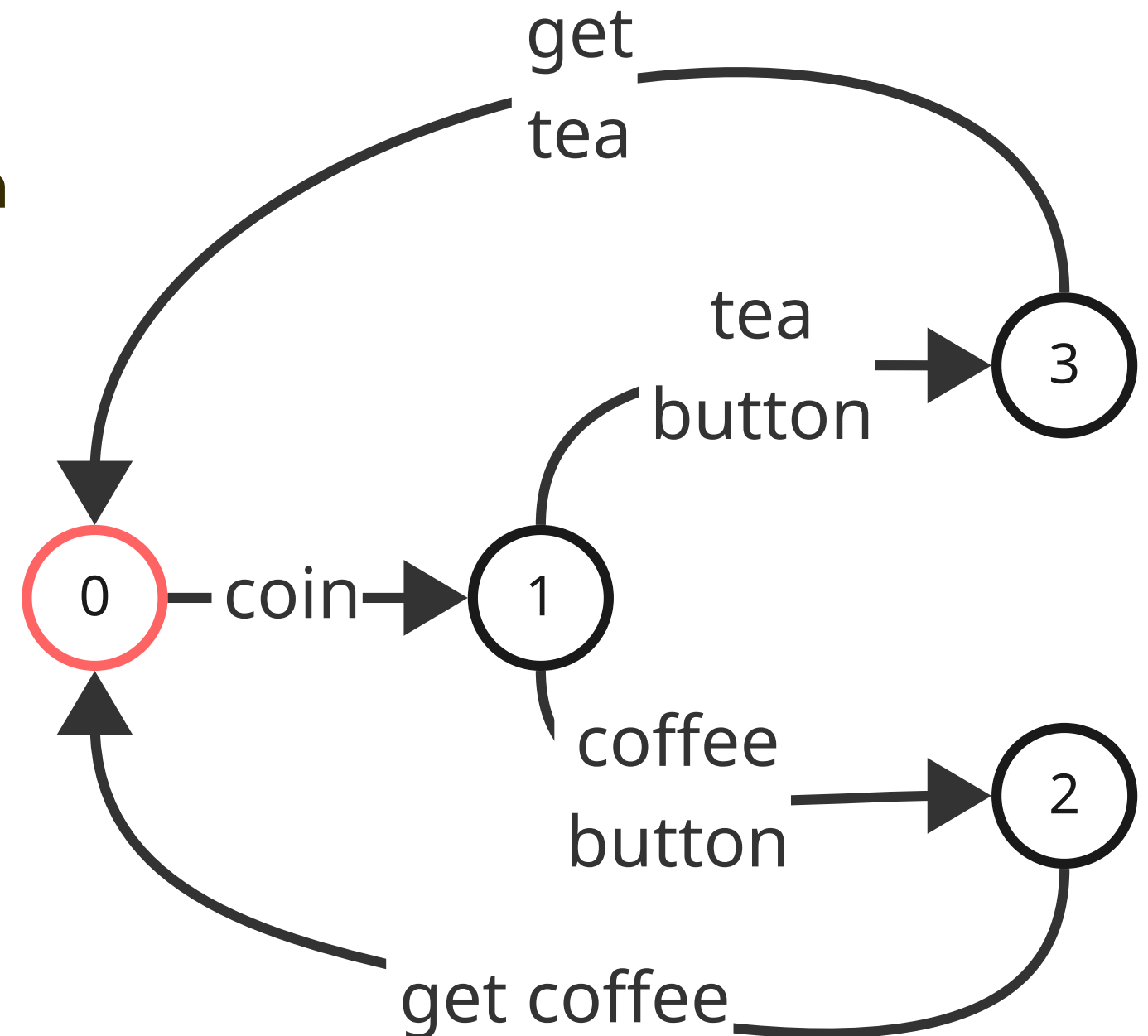
Client/Server or User/Application

In general, cases where two components can both act independently by also can interact with each others

(Labeled) Transition Systems

Used to model systems $(S, Act, \rightarrow, I)$

- S is a set of states
- Act is a set of labels
- $\rightarrow \subseteq S \times Act \times S$ is a transition relation
- I is the set of initial states



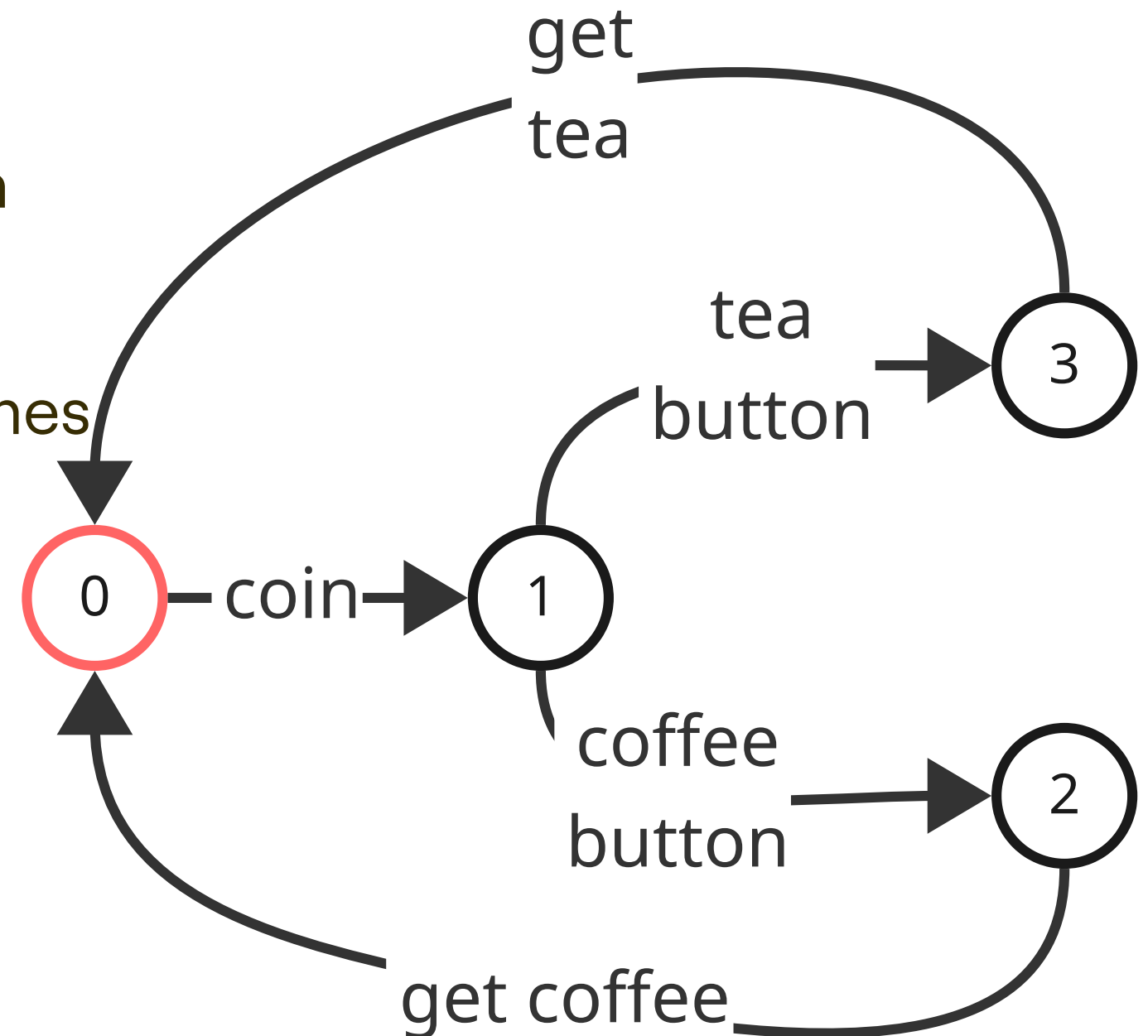
(Labeled) Transition Systems

Used to model systems $(S, Act, \rightarrow, I)$

- S is a set of states
- Act is a set of labels
- $\rightarrow \subseteq S \times Act \times S$ is a transition relation
- I is the set of initial states

arbitrariness choice of state and action names

τ usually represents an internal action



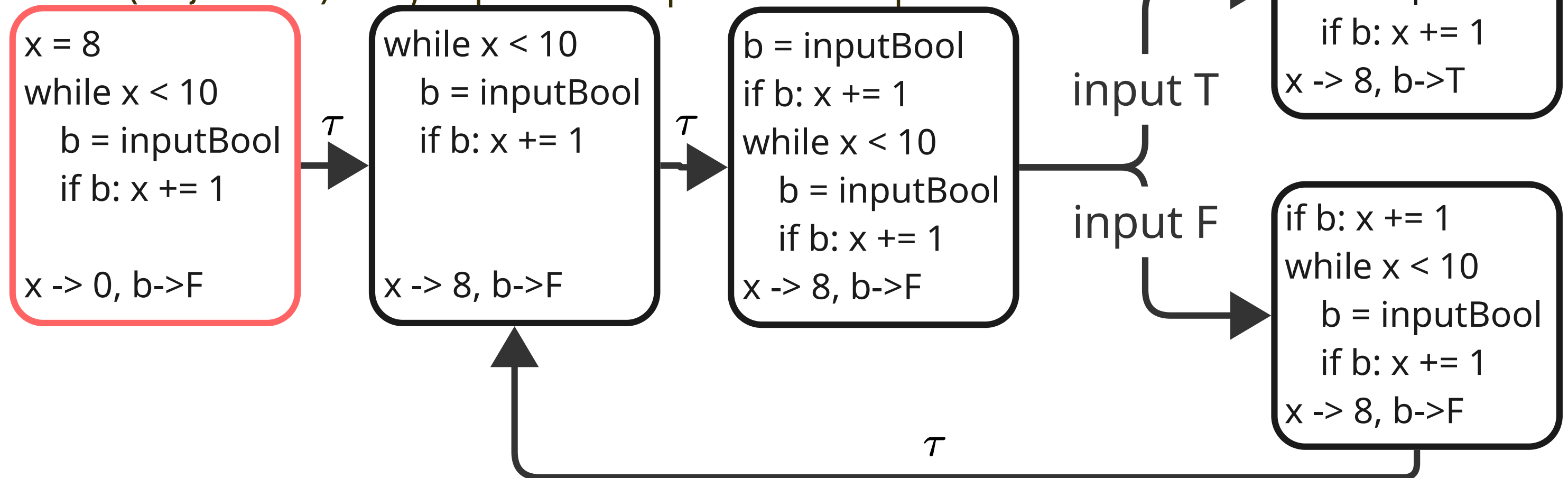
LTS example

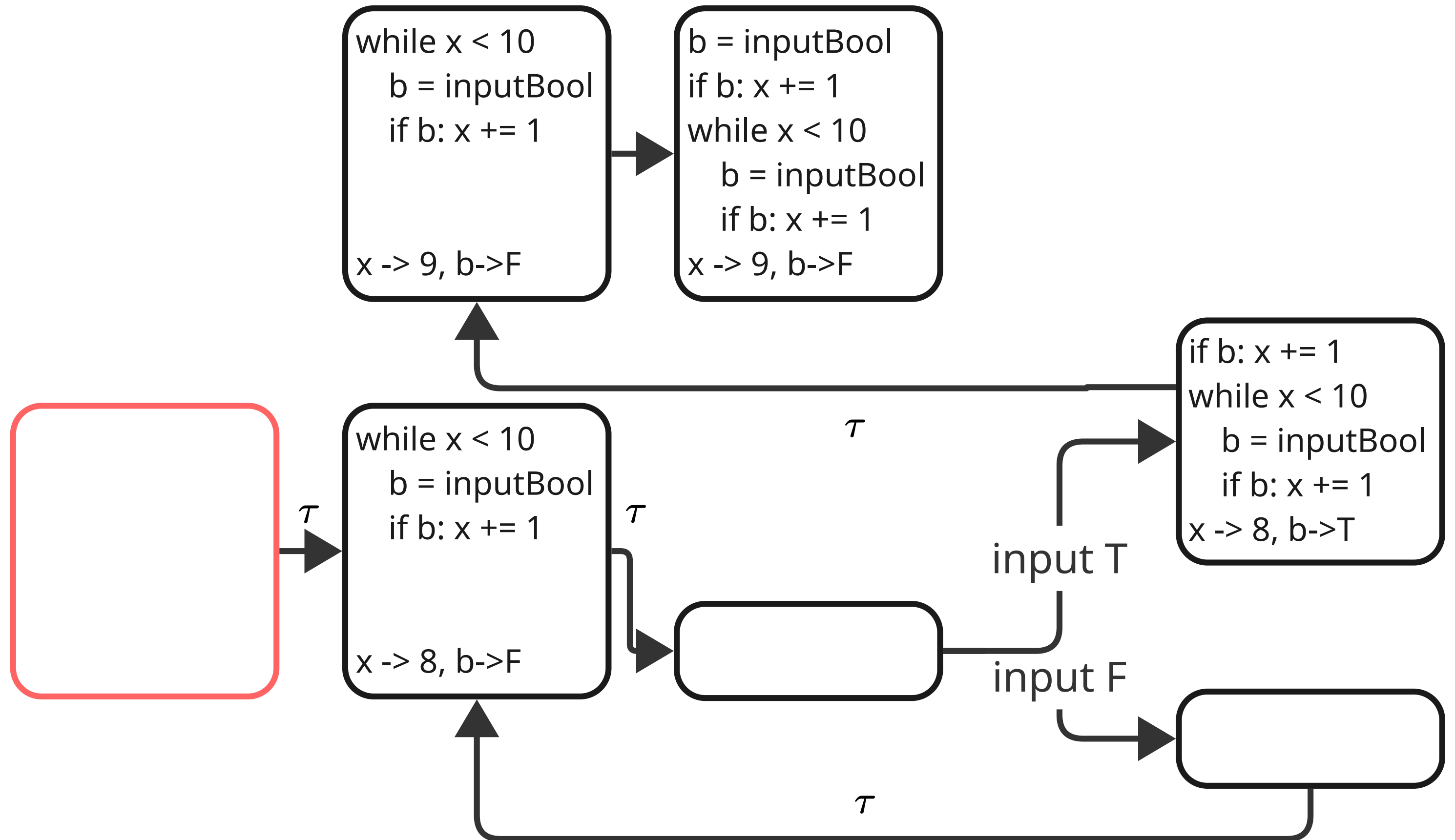
An LTS can be used to model the (small step) semantics of a program.

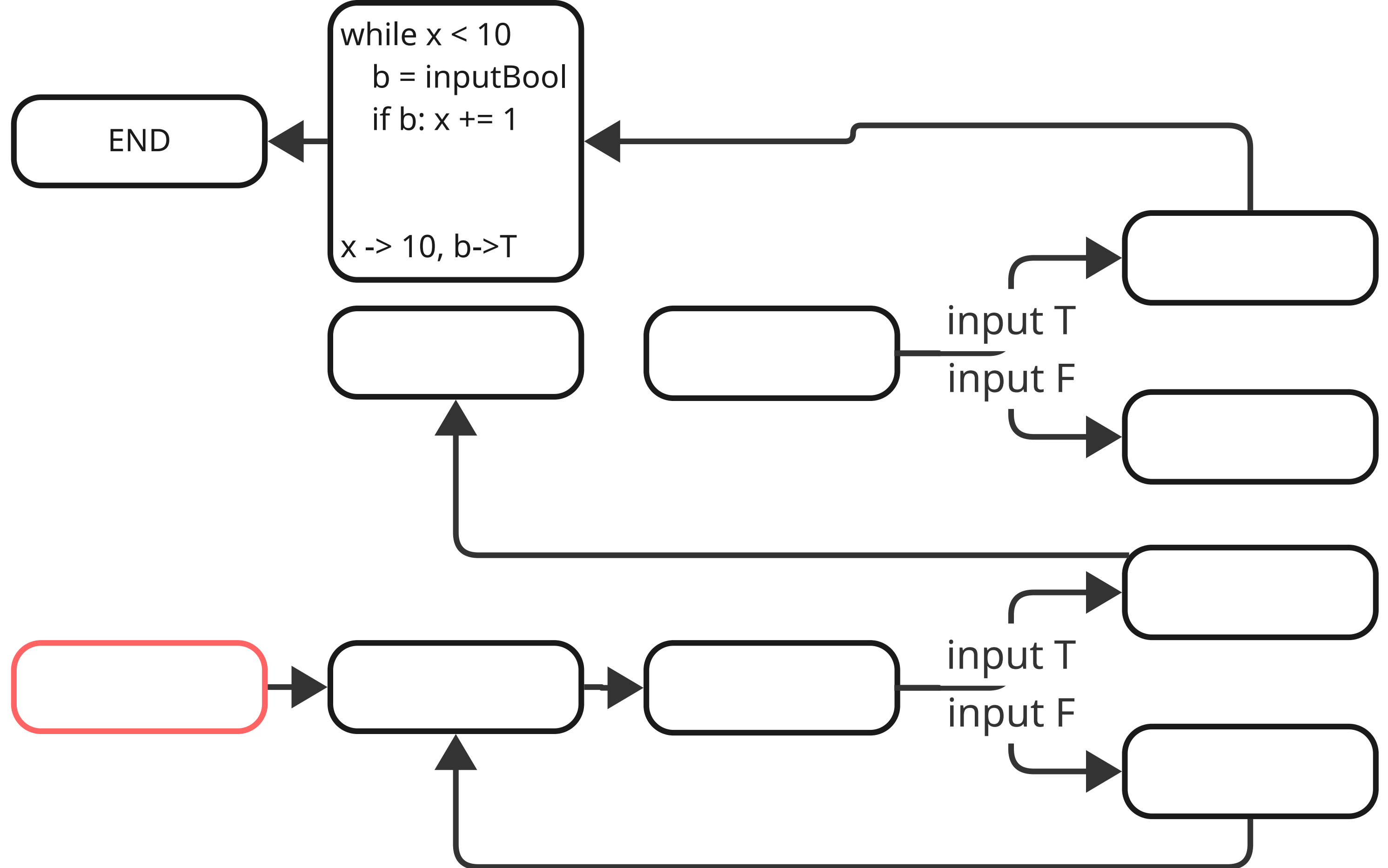
States reflect program's progress and variable assignments:

$$s = (p, m)$$

Actions (i.e., labels) may represent input and outputs







(Labeled) Transition Systems

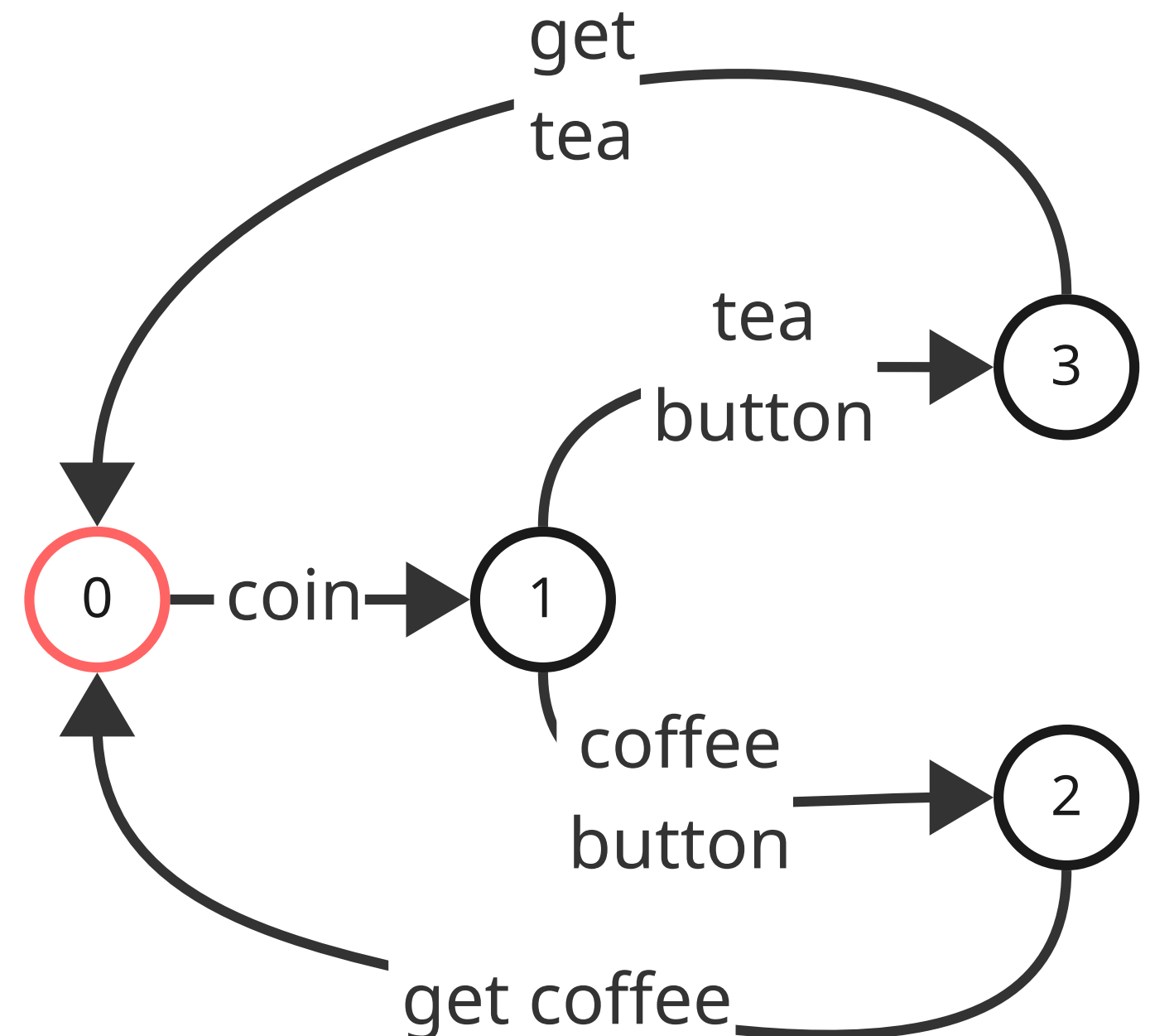
$(S, Act, \rightarrow, I)$

$\alpha \in Act, s \in S$. α -successors of s
 $post(s, \alpha) = \{s' \mid s \xrightarrow{\alpha} s'\}$

successors of s
 $post(s) = \cup_{\alpha \in Act} post(s, \alpha)$

predecessors
 $pre(s, \alpha) = \{s' \mid s' \xrightarrow{\alpha} s\}$
 $pre(s) = \cup_{\alpha \in Act} pre(s, \alpha)$

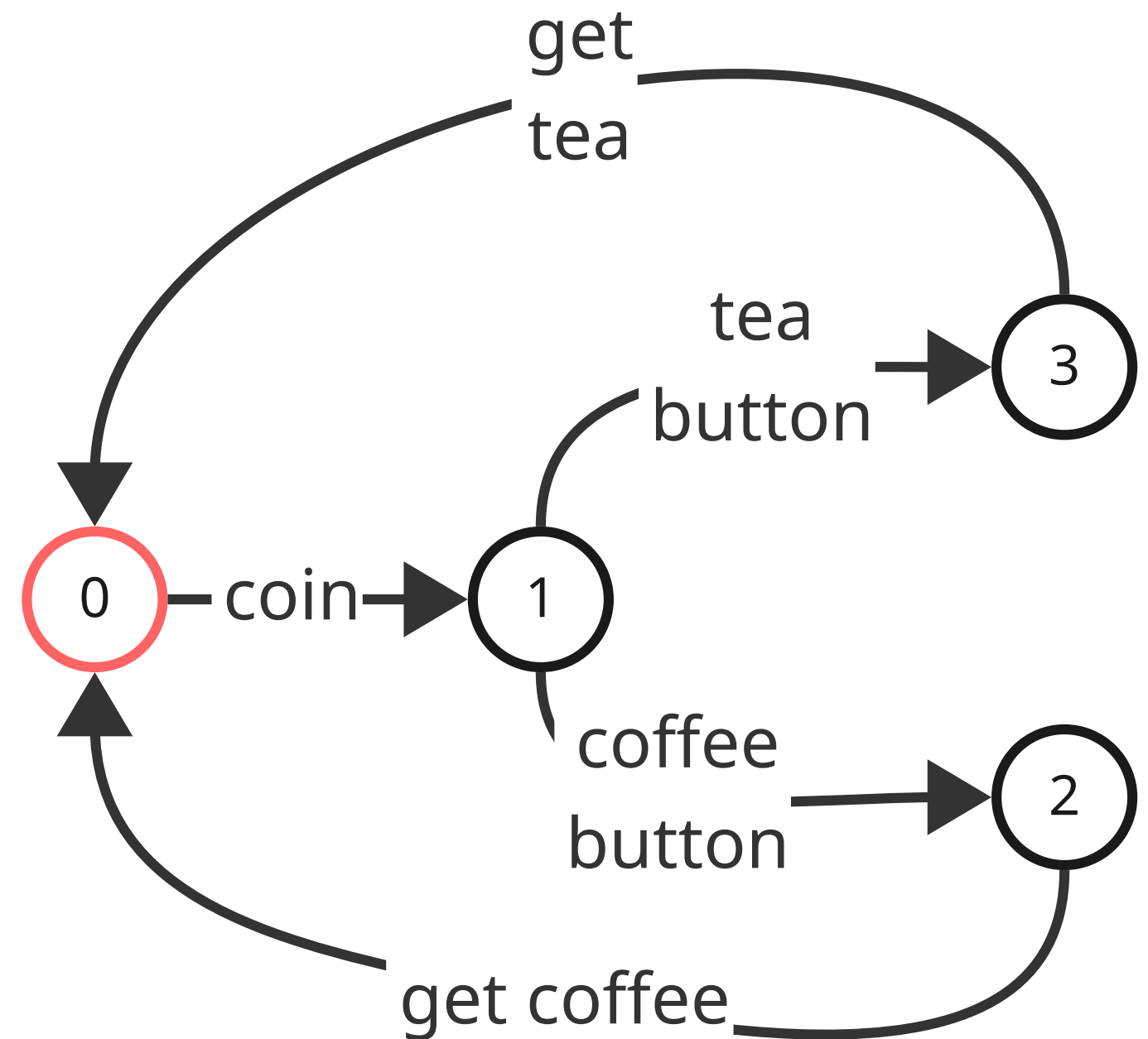
state s is terminal if
 $post(s) = \emptyset$



(Labeled) Transition Systems

An LTS is deterministic iff

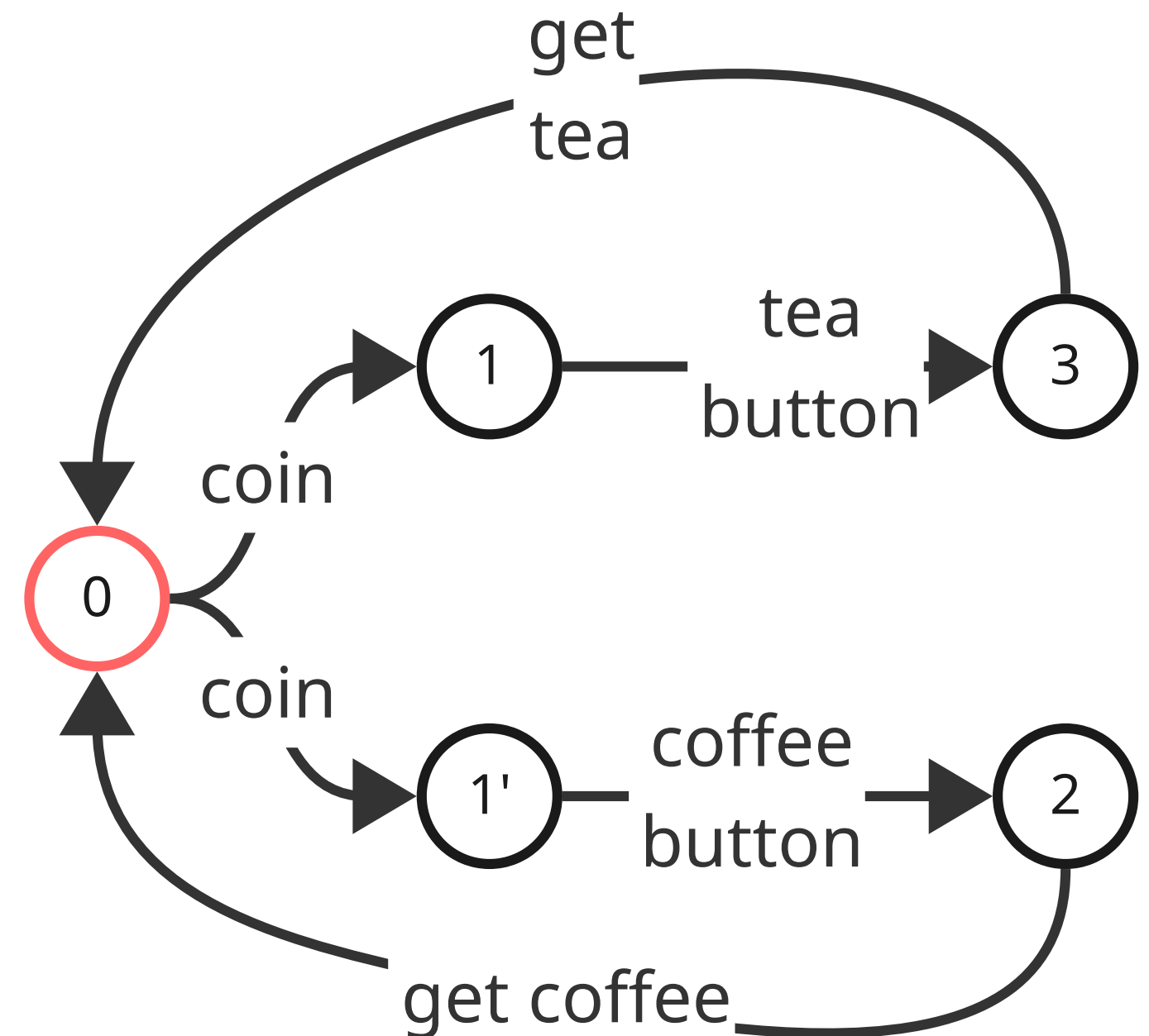
$$|I| \leq 1 \wedge \forall s, \alpha. |post(s, \alpha)| \leq 1$$



(Labeled) Transition Systems

An LTS is deterministic iff

$$|I| \leq 1 \wedge \forall s, \alpha. |post(s, \alpha)| \leq 1$$



Executions

An finite execution fragment is a finite sequence

$$\rho = s_1 \alpha_1 s_2 \alpha_2 \dots s_n$$

such that $s_i \xrightarrow{\alpha_i} s_{i+1}$

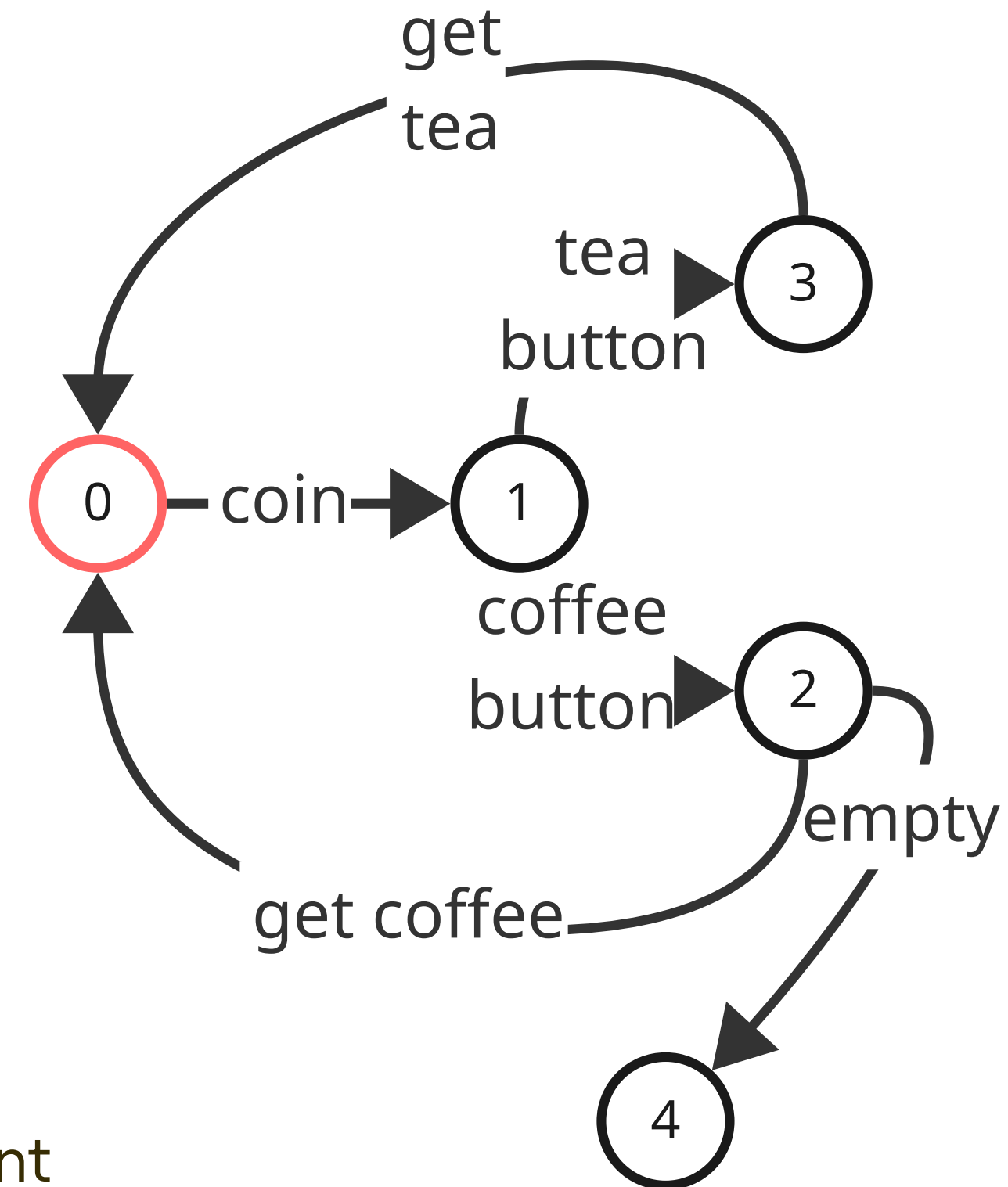
Any state is a valid singleton execution fragment

An execution fragment can be finite or infinite

A maximal execution fragment cannot be extended
Either is infinite or the last state is final

An execution fragment is initial if its first state
is initial

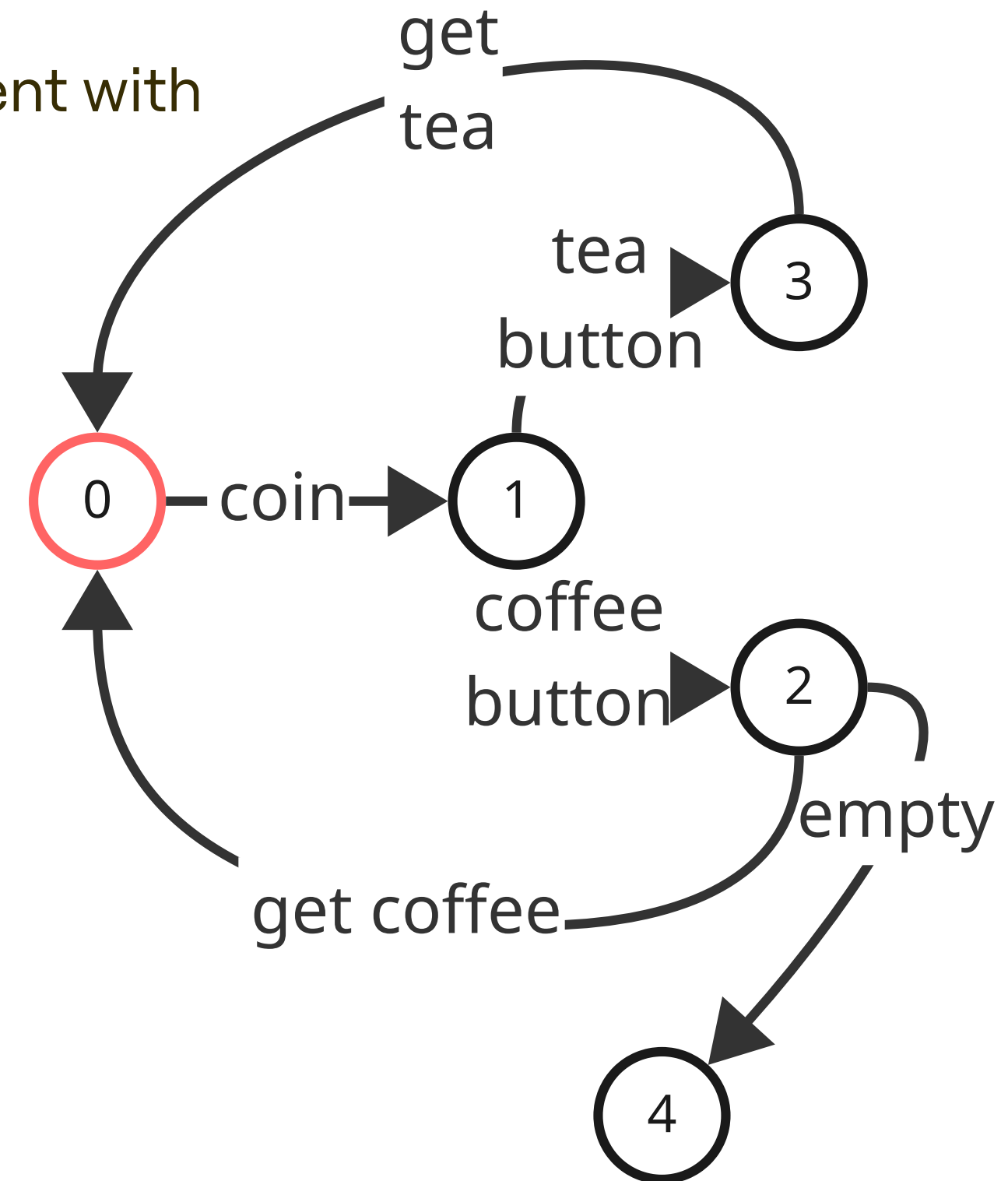
An execution is an initial maximal execution fragment



Reachable states

States s such that there is a initial execution fragment with

$$s_1\alpha_1s_2\alpha_2\dots s$$



Interleaved Semantics

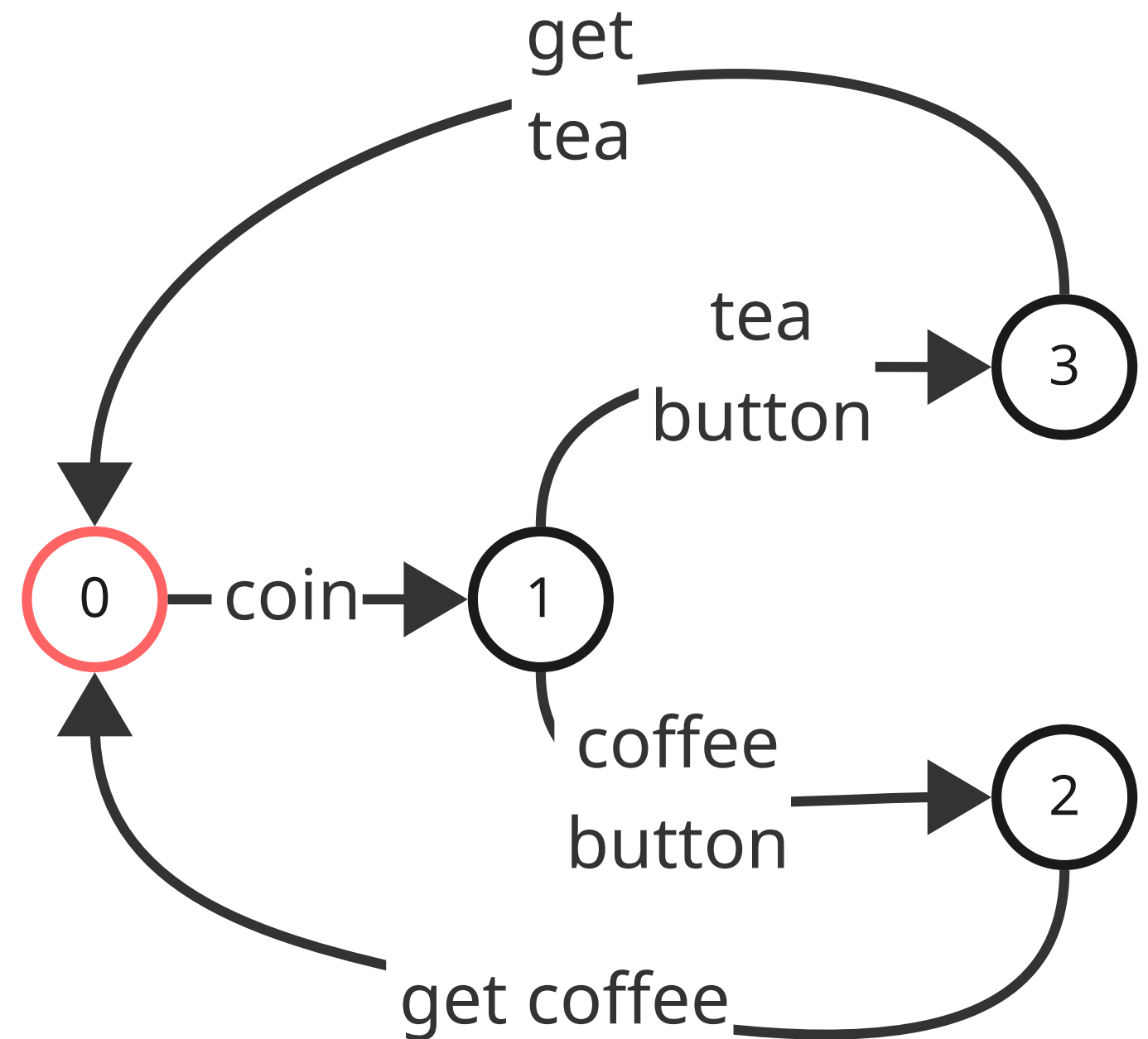
$$M_1 \parallel M_2$$

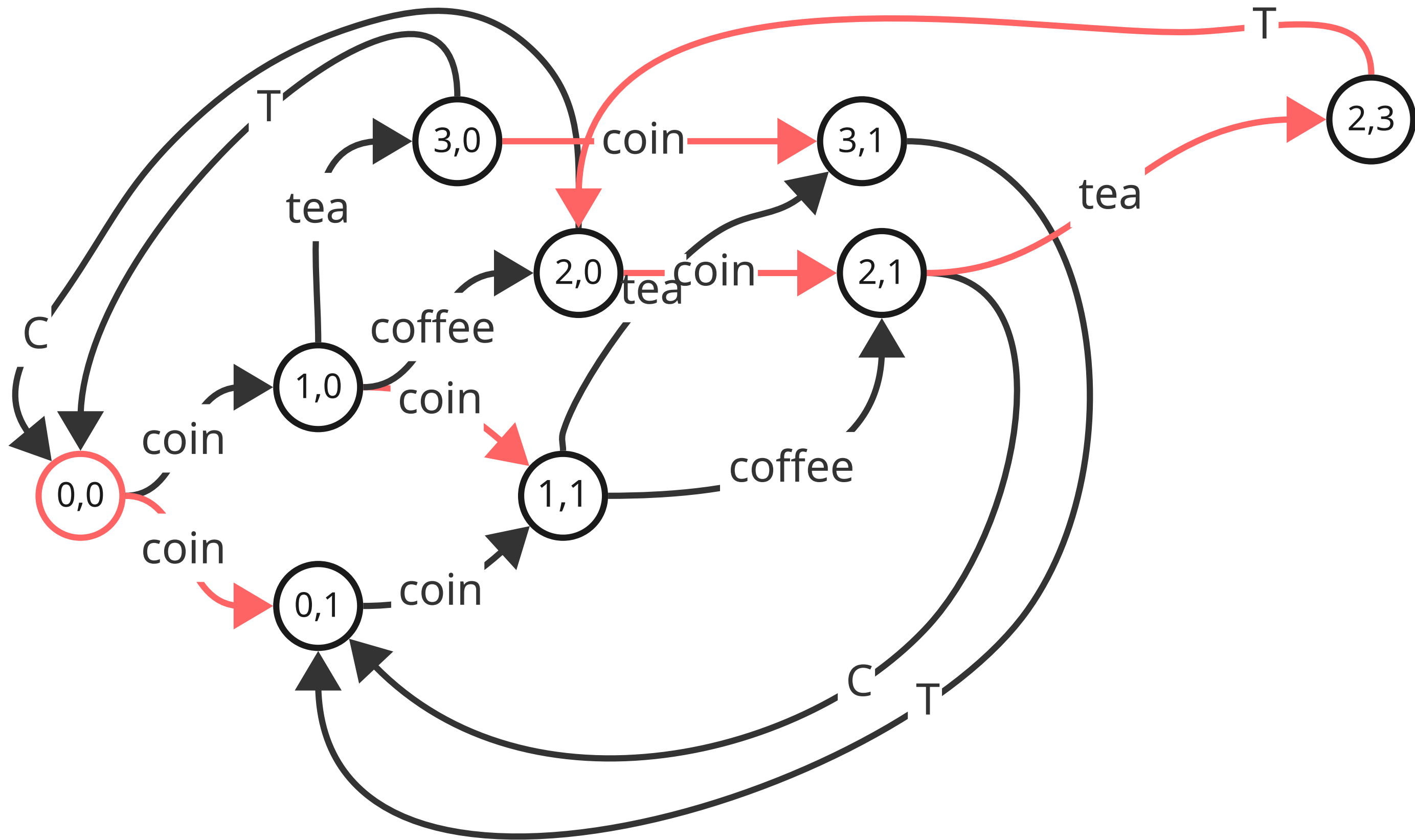
$$(S_1 \times S_2, Act_1 \cup Act_2, \rightarrow', I_1 \times I_2)$$

$$\frac{s_1 \xrightarrow{\alpha} s'_1}{s_1 \parallel s_2 \xrightarrow{\alpha} s'_1 \parallel s_2}$$

$$\frac{s_2 \xrightarrow{\alpha} s'_2}{s_1 \parallel s_2 \xrightarrow{\alpha} s_1 \parallel s'_2}$$

Models independent systems



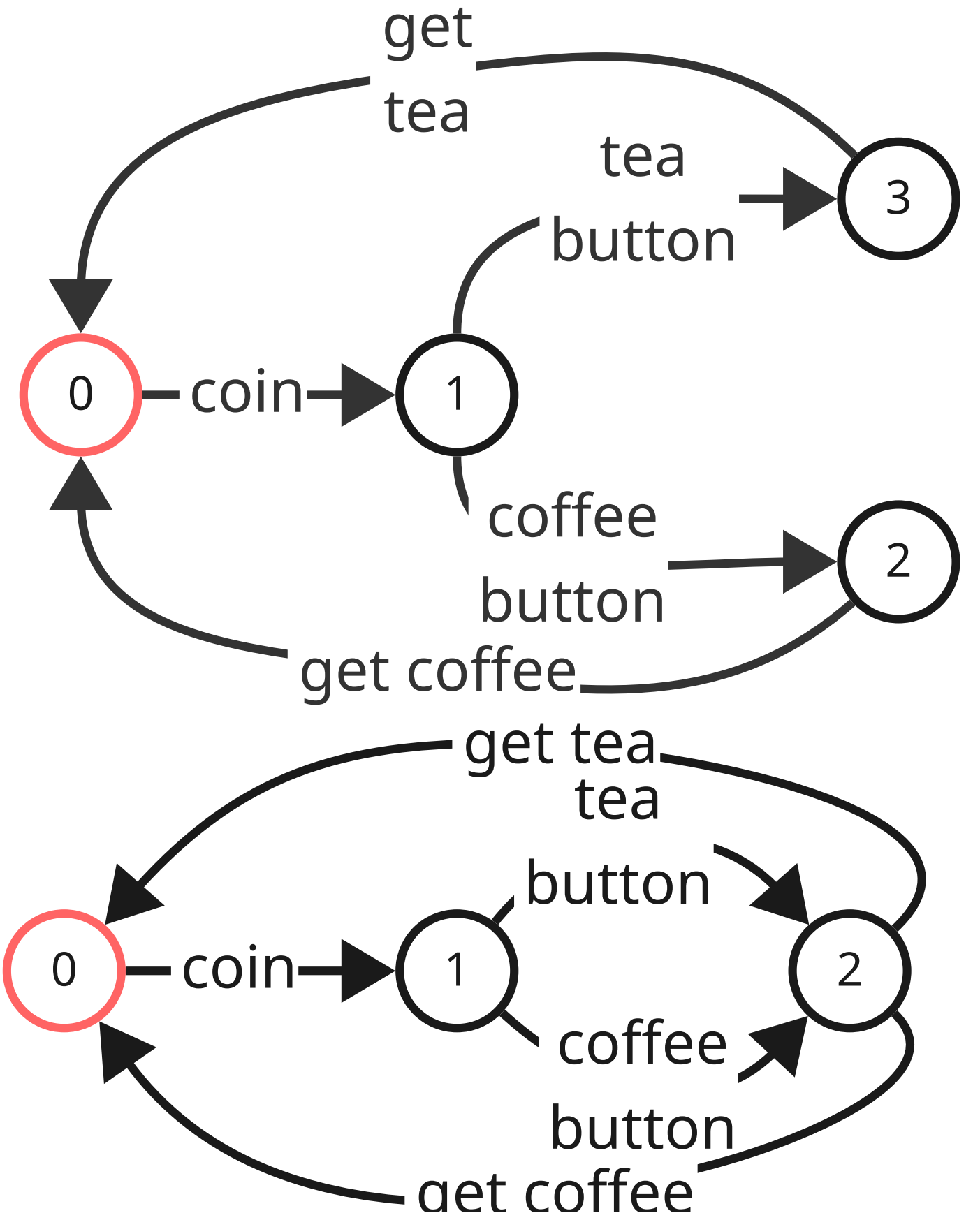


Synchronous Composition

$$(S_1 \times S_2, Act_1 \cup Act_2, \rightarrow', I_1 \times I_2)$$

$$\frac{s_1 \xrightarrow{\tau} s'_1}{s_1 \parallel s_2 \xrightarrow{\tau} s'_1 \parallel s_2} \quad \frac{s_2 \xrightarrow{\tau} s'_2}{s_1 \parallel s_2 \xrightarrow{\tau} s_1 \parallel s'_2}$$

$$\frac{s_1 \xrightarrow{\alpha} s'_1 \wedge s_2 \xrightarrow{\alpha} s'_2}{s_1 \parallel s_2 \xrightarrow{\alpha} s'_1 \parallel s'_2}$$

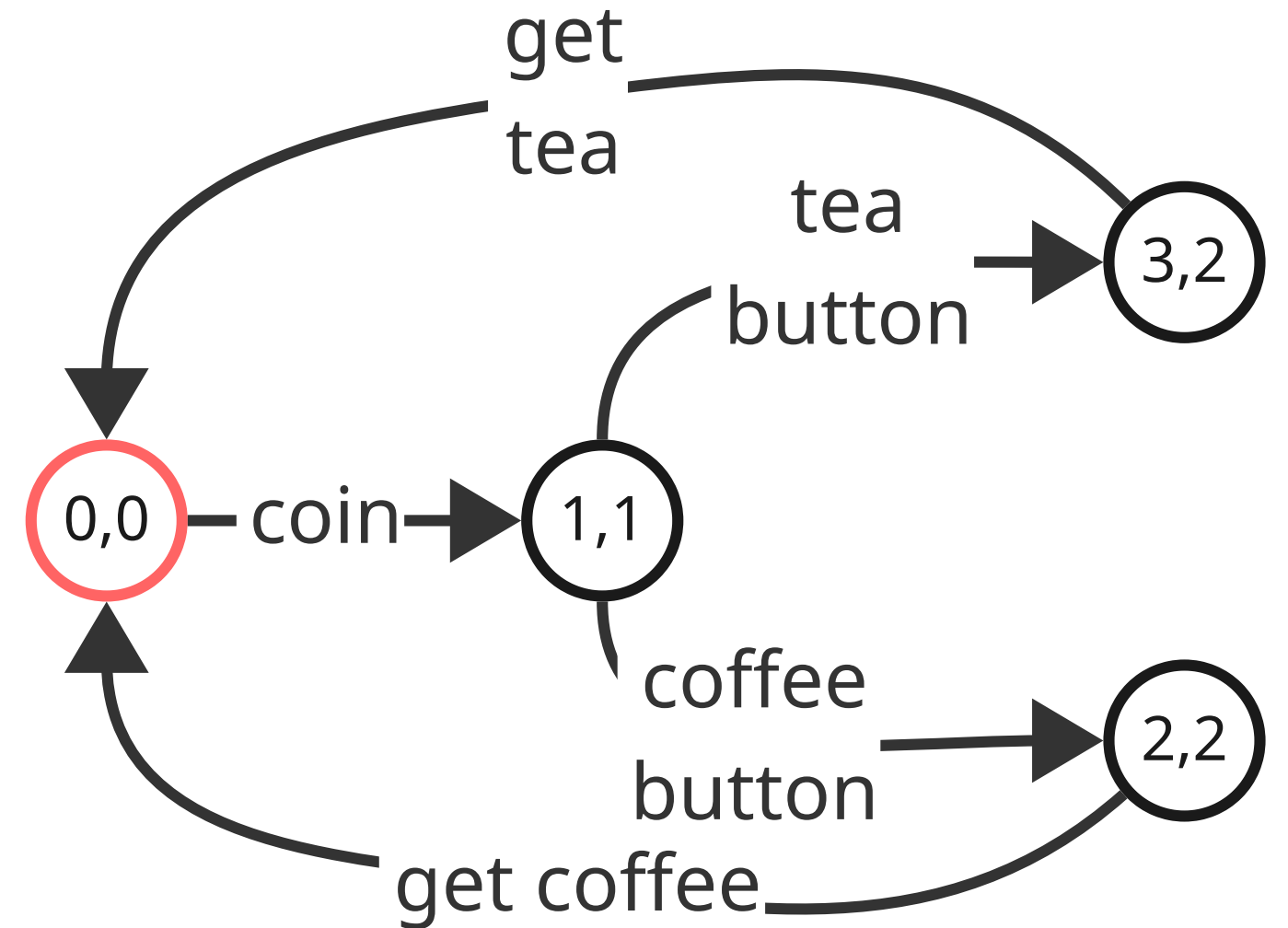


Synchronous Composition

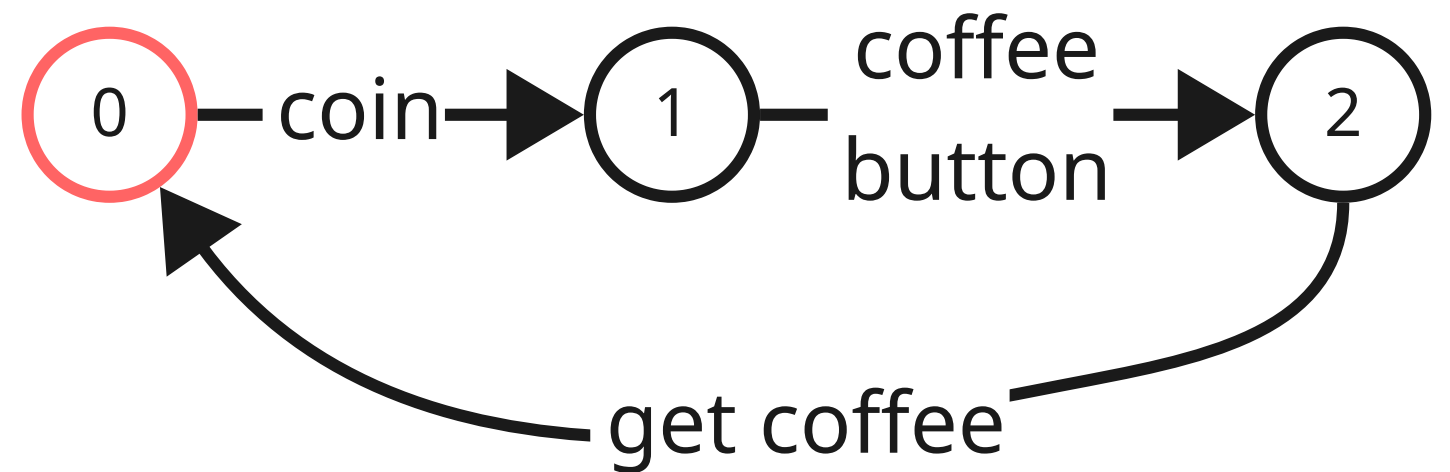
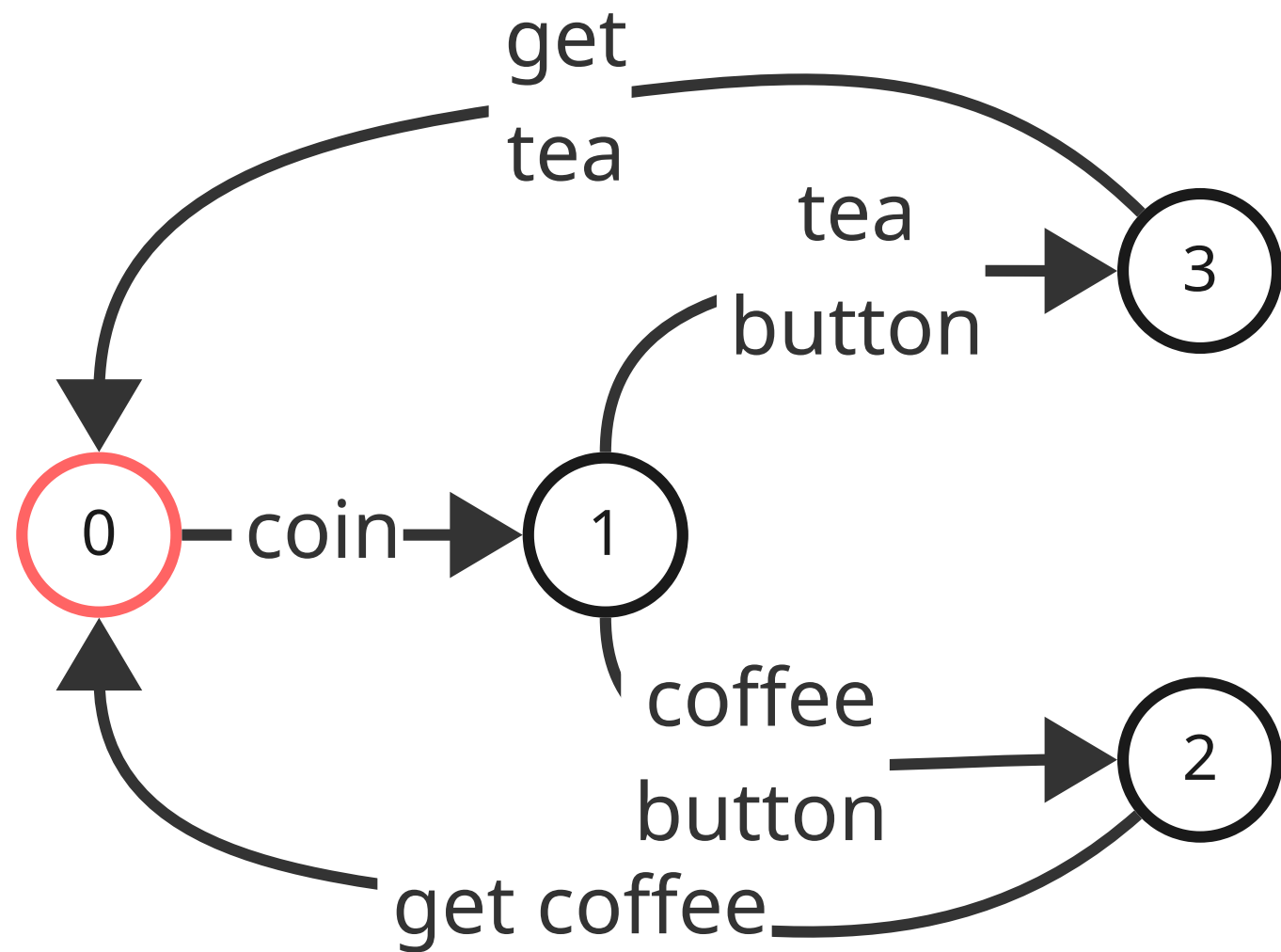
$$(S_1 \times S_2, Act_1 \cup Act_2, \rightarrow', I_1 \times I_2)$$

$$\frac{s_1 \xrightarrow{\tau} s'_1}{s_1 \parallel s_2 \xrightarrow{\tau} s'_1 \parallel s_2} \qquad \frac{s_2 \xrightarrow{\tau} s'_2}{s_1 \parallel s_2 \xrightarrow{\tau} s_1 \parallel s'_2}$$

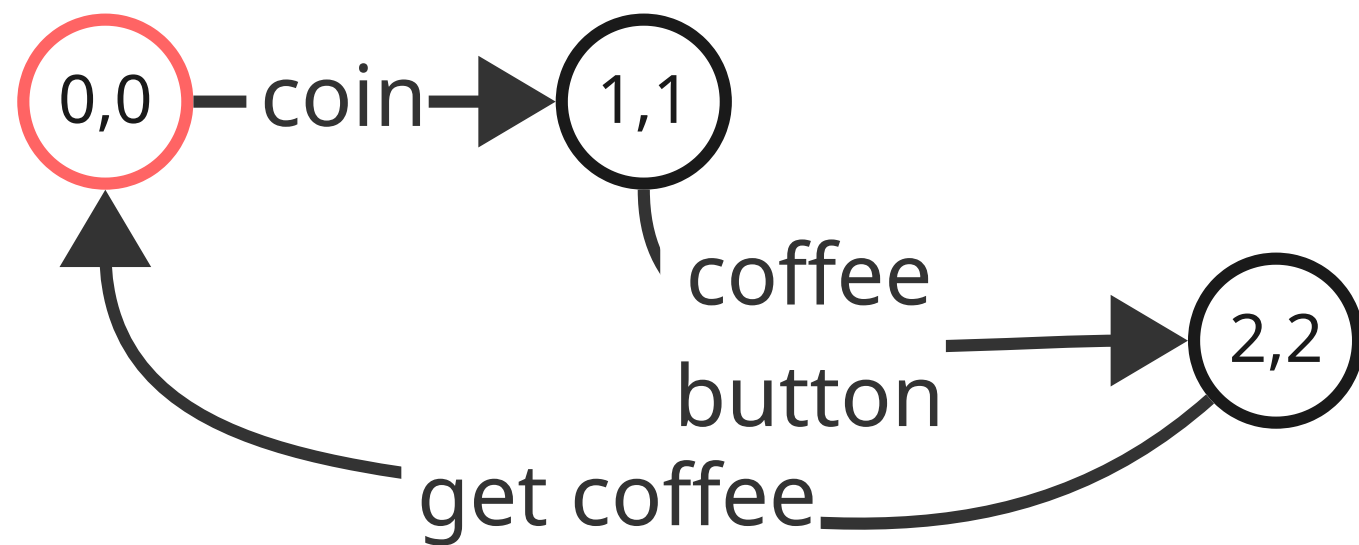
$$\frac{s_1 \xrightarrow{\alpha} s'_1 \wedge s_2 \xrightarrow{\alpha} s'_2}{s_1 \parallel s_2 \xrightarrow{\alpha} s'_1 \parallel s'_2}$$



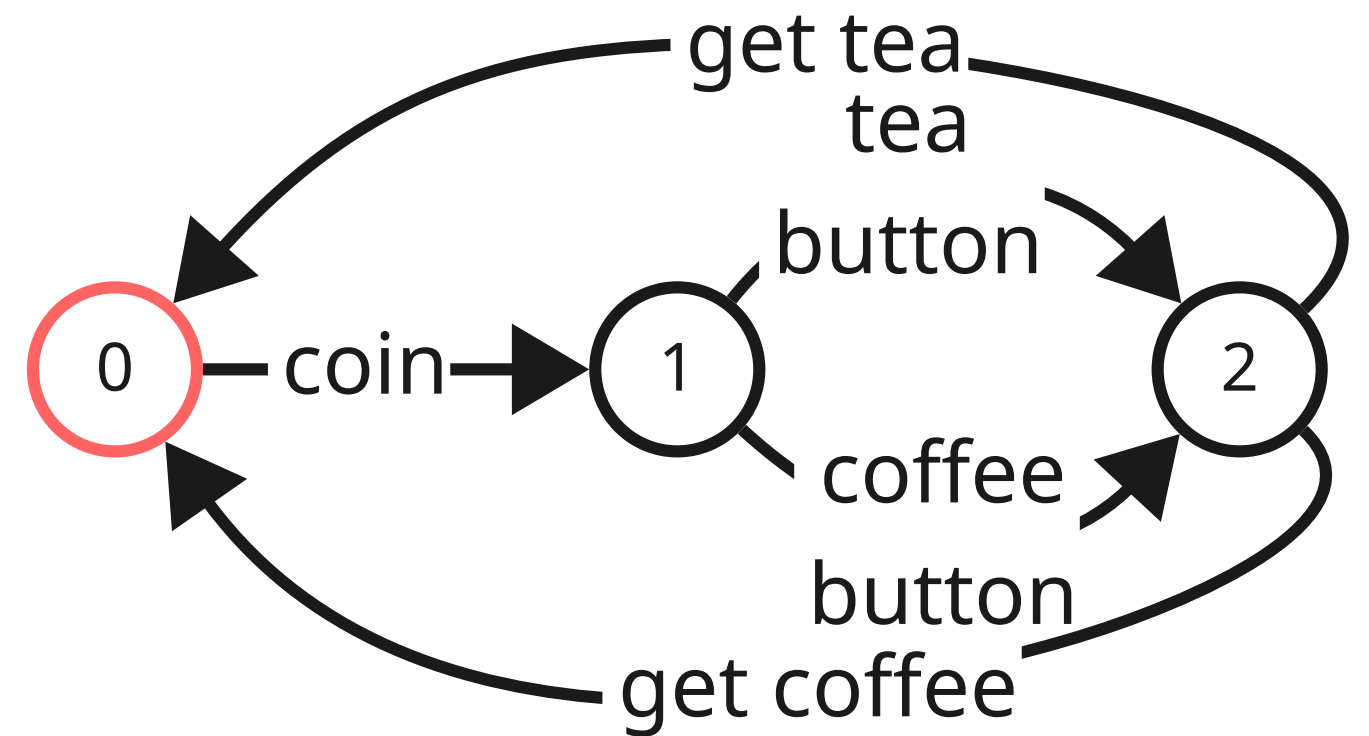
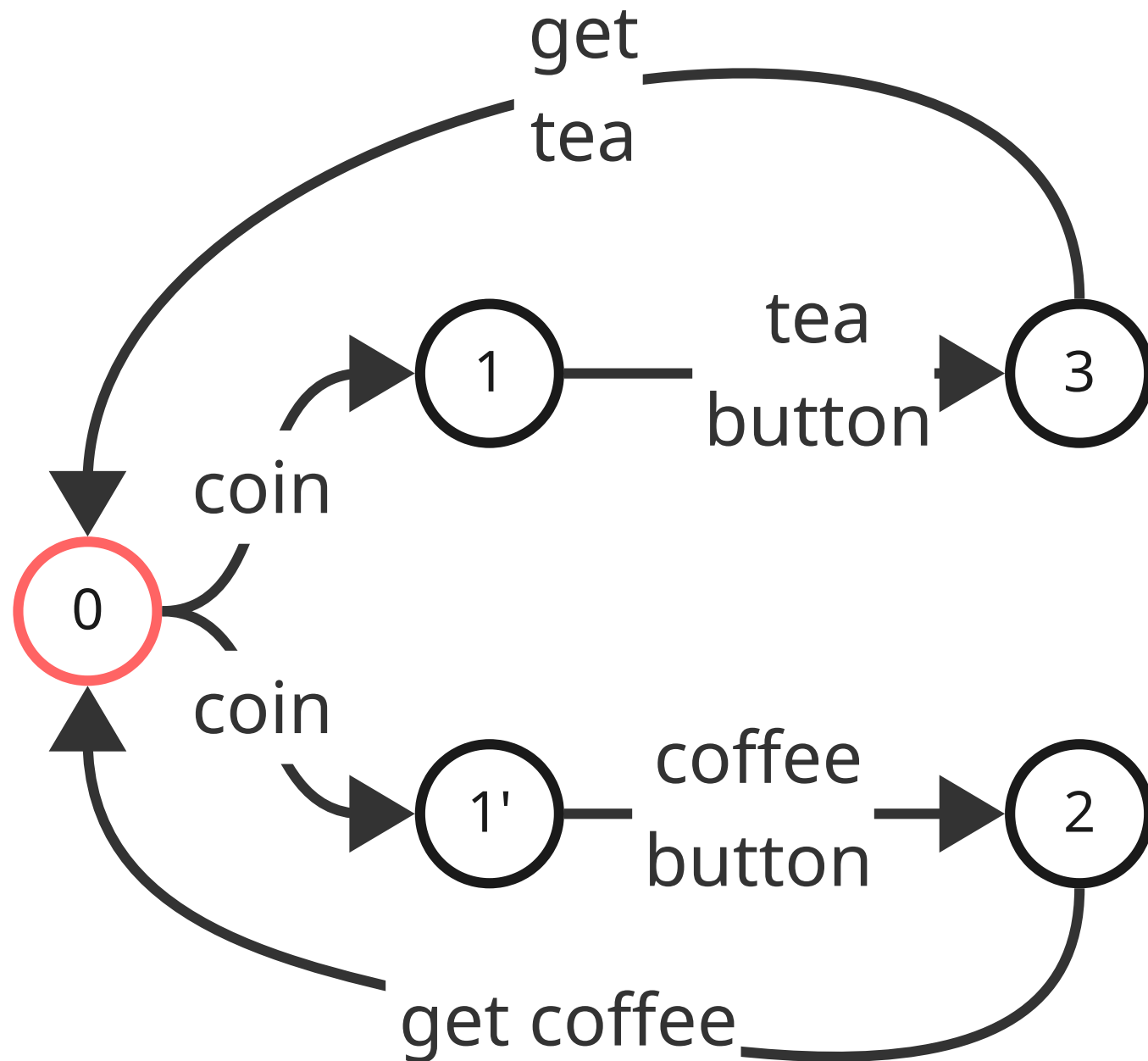
Synchronous Composition



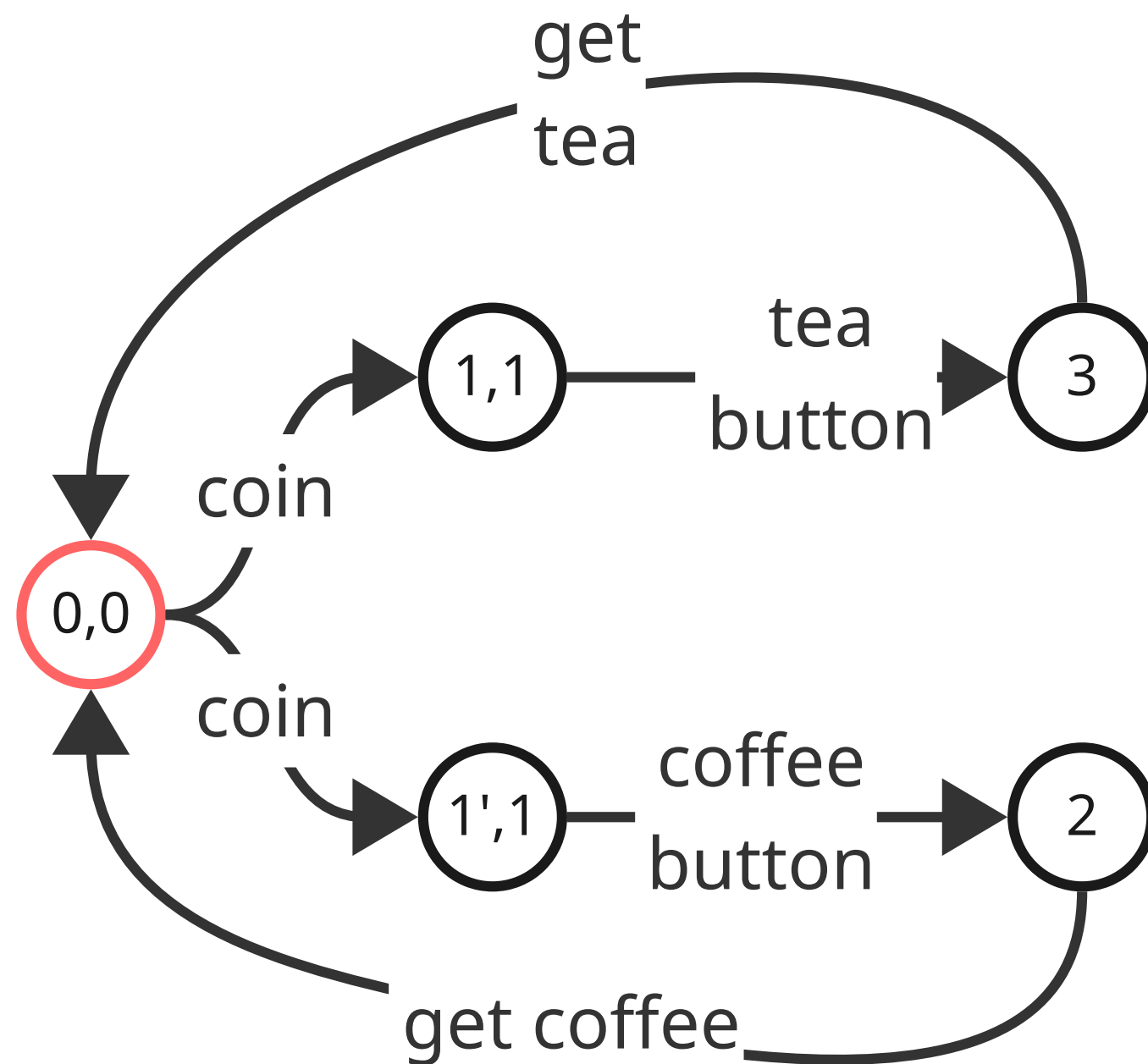
Synchronous Composition



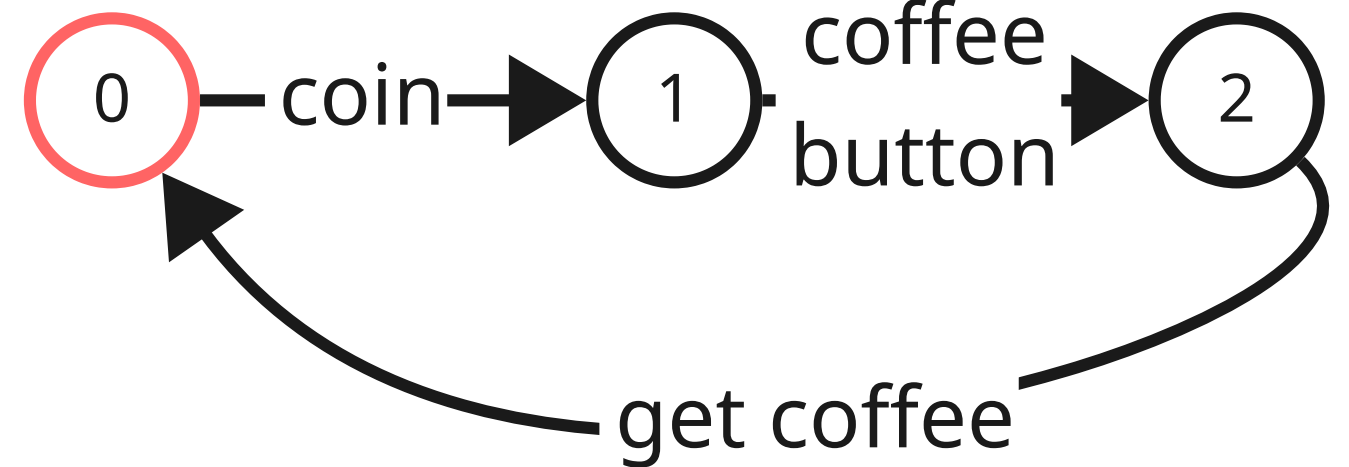
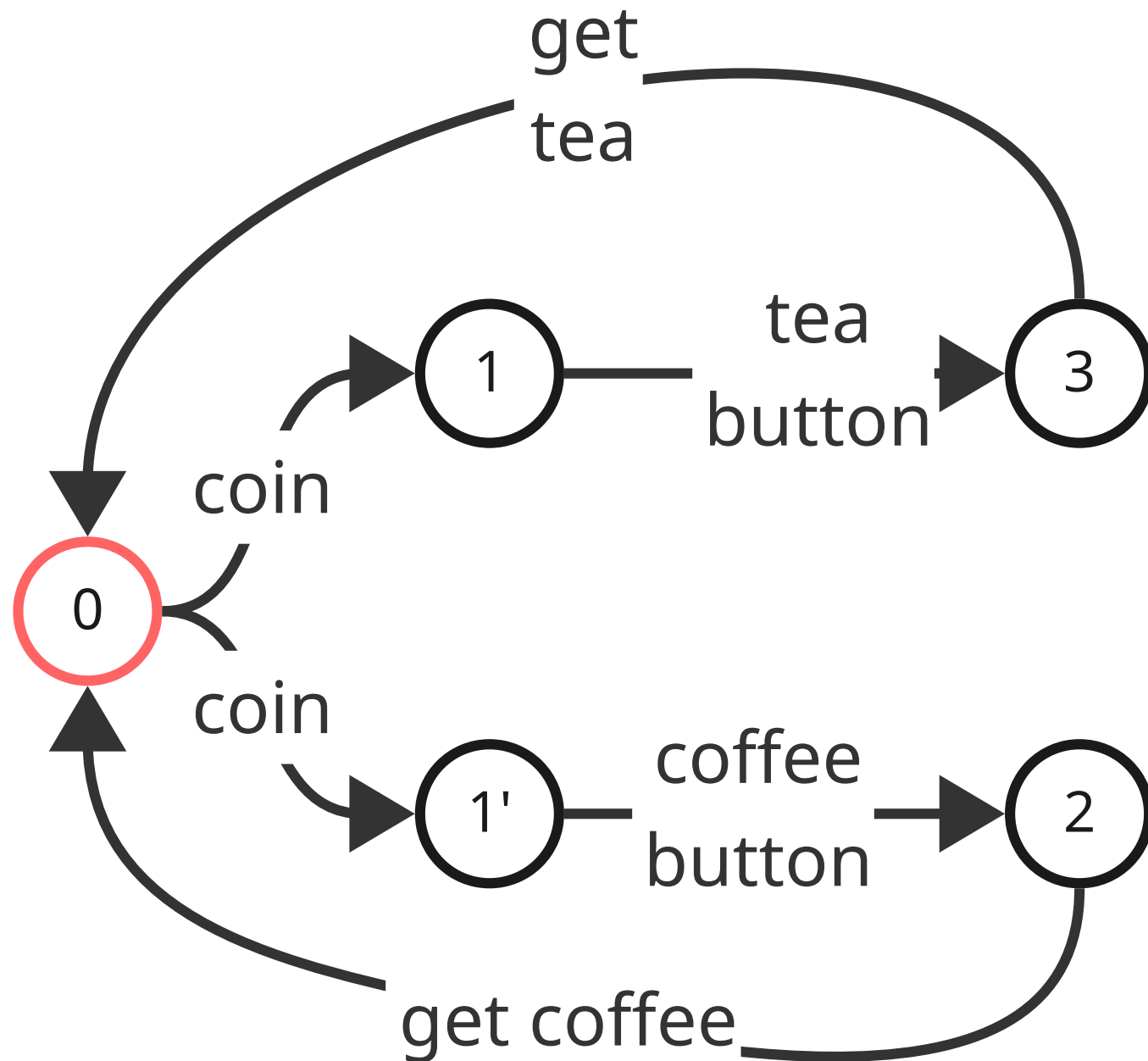
Synchronous Composition



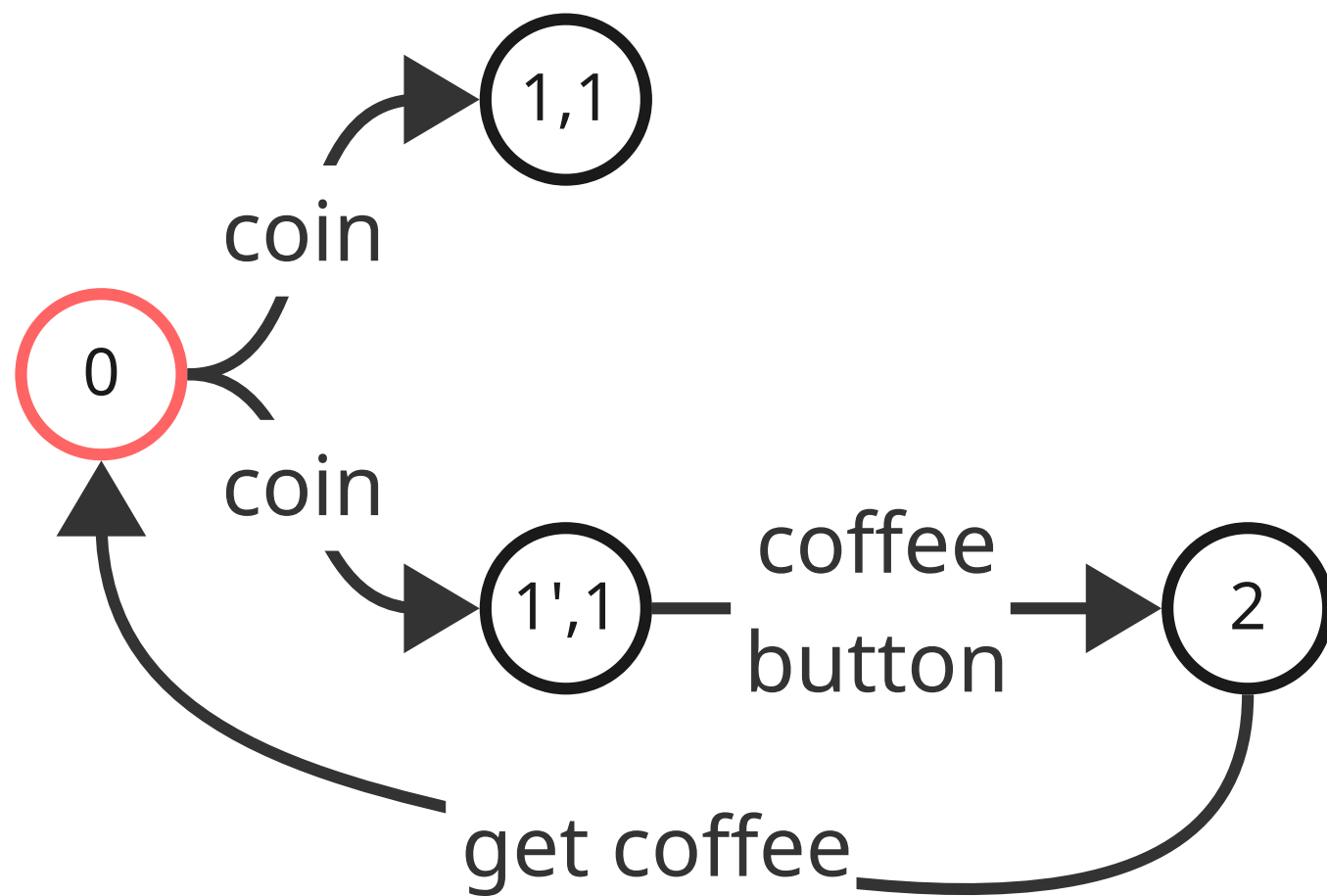
Synchronous Composition



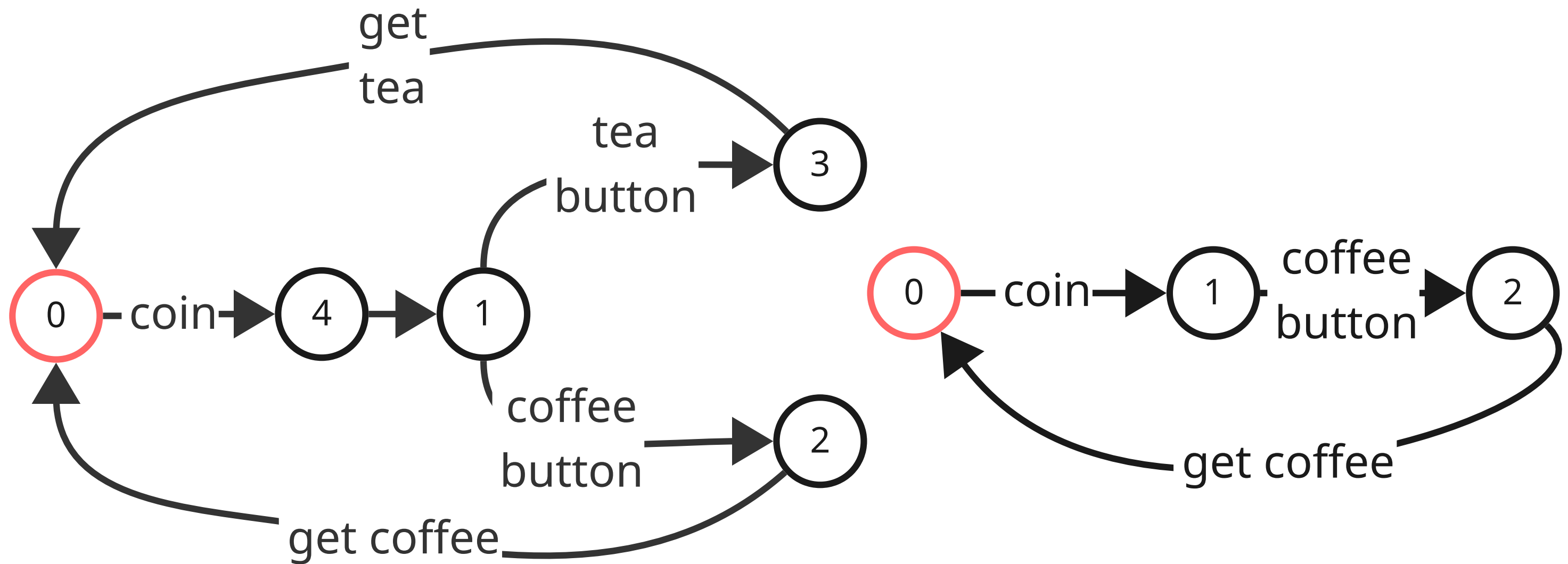
Synchronous Composition



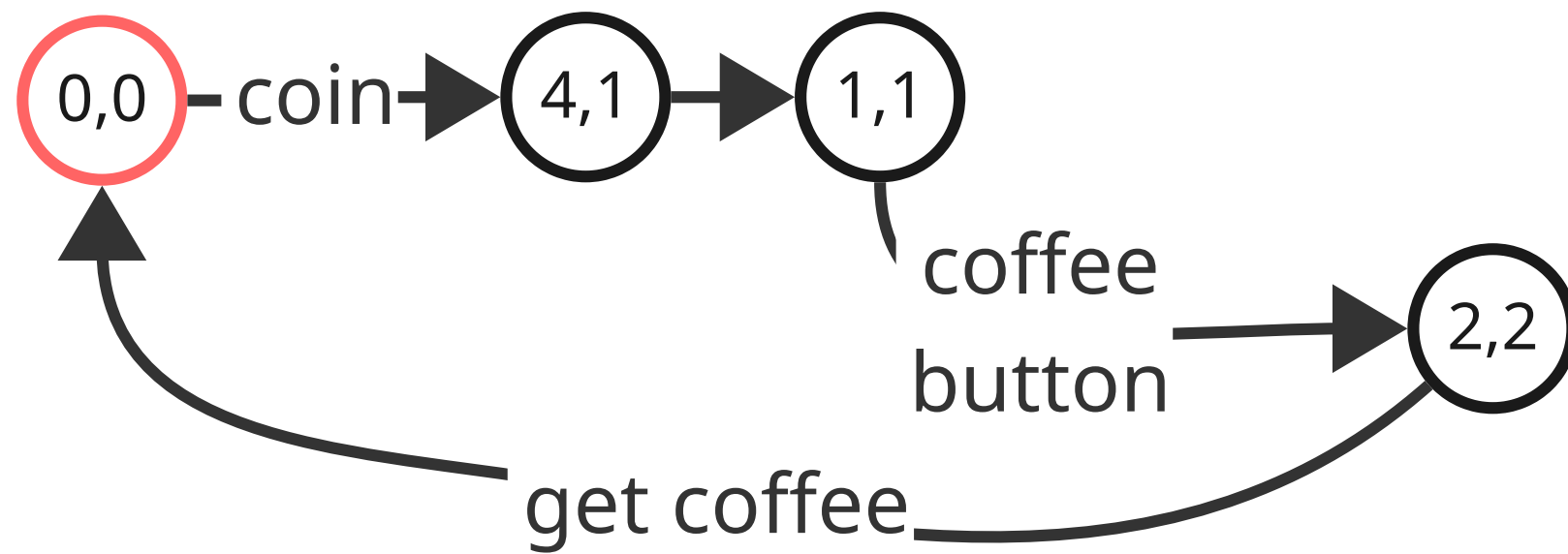
Synchronous Composition



Synchronous Composition



Synchronous Composition



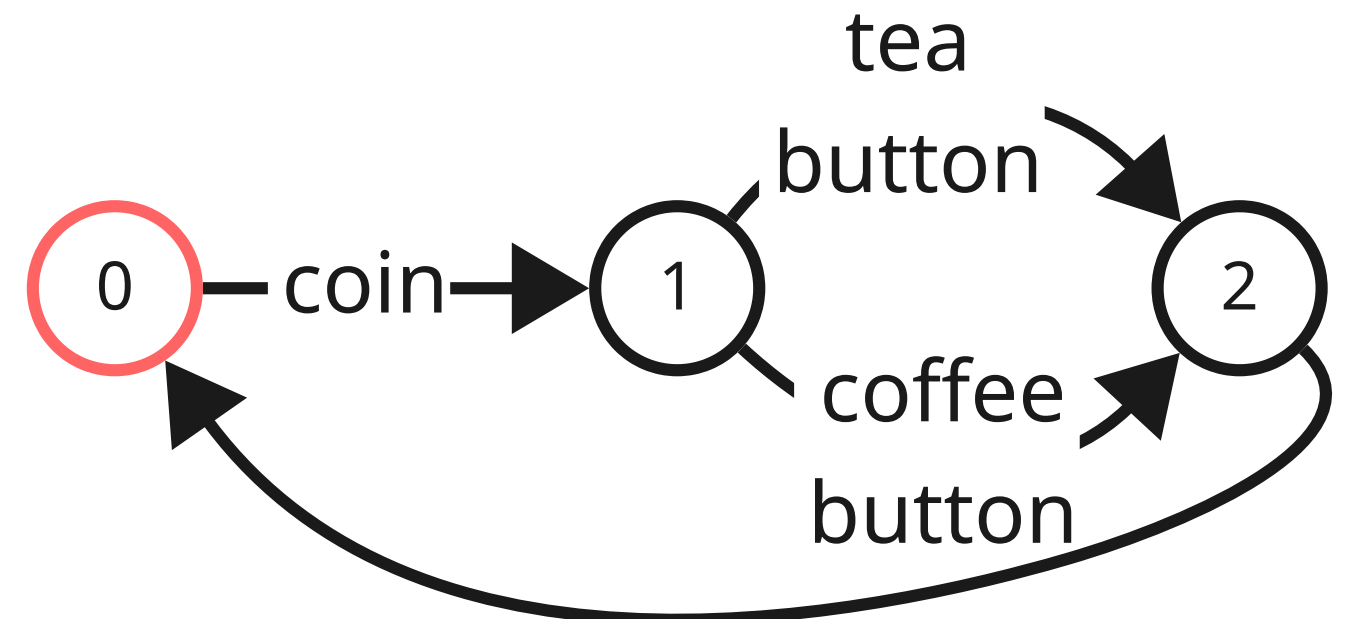
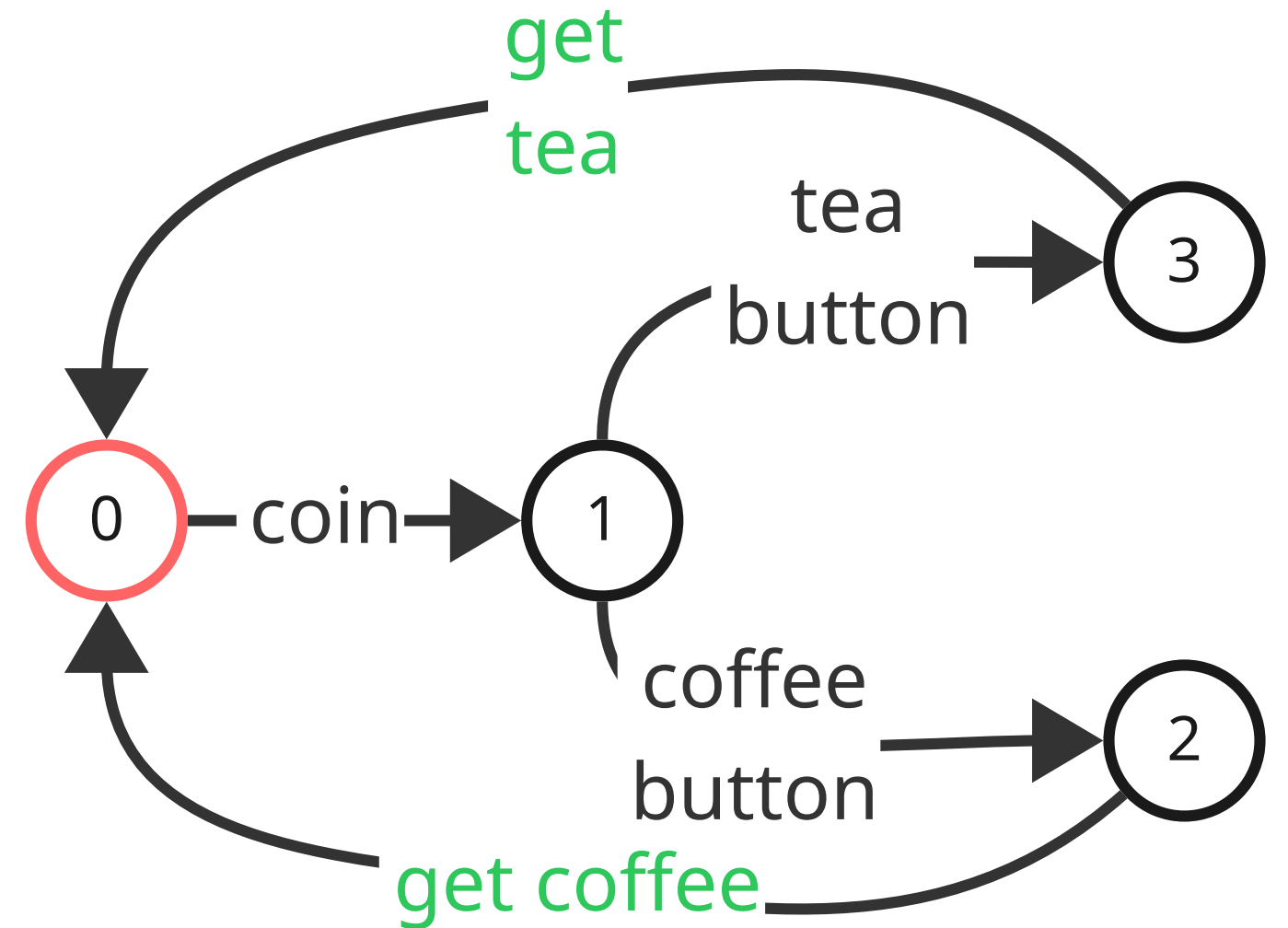
Synchronous Composition

$$(S_1 \times S_2, Act_1 \cup Act_2, \rightarrow', I_1 \times I_2)$$

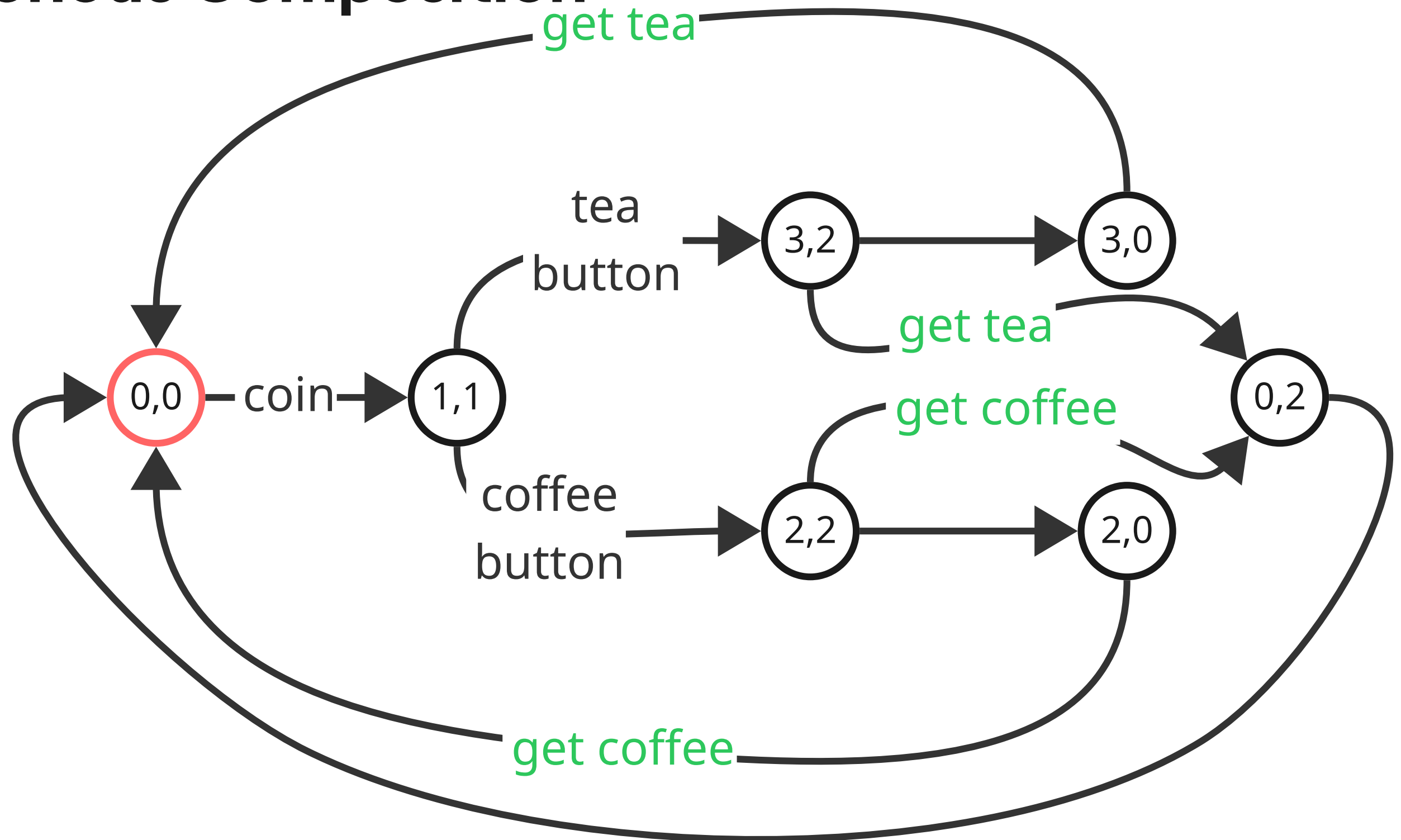
$$H \subseteq Act_1 \cap Act_2$$

$$\frac{s_1 \xrightarrow{\alpha} s'_1 \wedge \alpha \notin H}{s_1 \parallel s_2 \xrightarrow{\alpha} s'_1 \parallel s_2} \quad \frac{s_2 \xrightarrow{\alpha} s'_2 \wedge \alpha \notin H}{s_1 \parallel s_2 \xrightarrow{\alpha} s_1 \parallel s'_2}$$

$$\frac{s_1 \xrightarrow{\alpha} s'_1 \wedge s_2 \xrightarrow{\alpha} s'_2 \wedge \alpha \in H}{s_1 \parallel s_2 \xrightarrow{\alpha} s'_1 \parallel s'_2}$$



Synchronous Composition



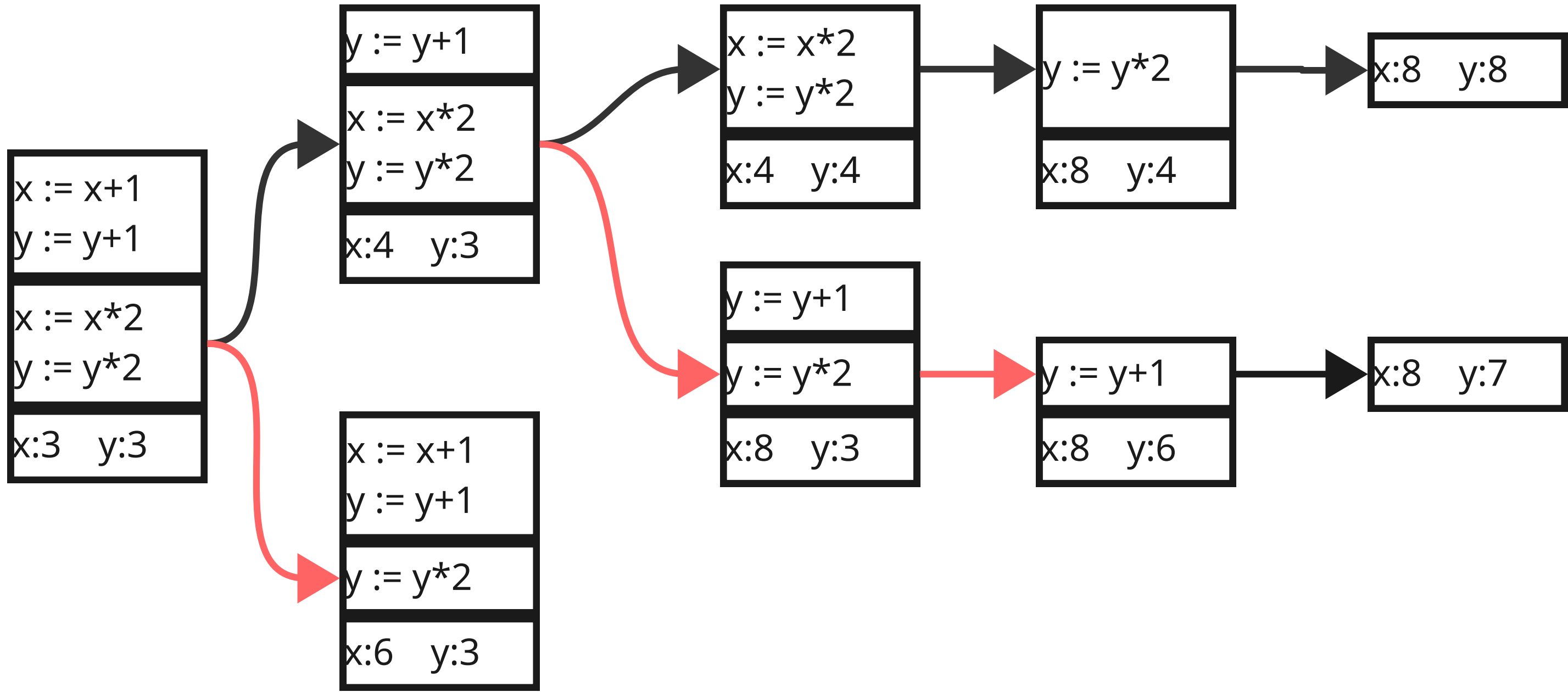
Shared Memory

$x := x+1$
 $y := y+1$

$x := x*2$
 $y := y*2$

$$\frac{p_1, m \rightarrow p'_1, m'}{p_1, p_2, m \rightarrow p'_1, p_2, m'}$$

$$\frac{p_2, m \rightarrow p'_2, m'}{p_1, p_2, m \rightarrow p_1, p'_2, m'}$$



Peeterson Agorithm

code executed by thread i [0,1]:

```
b[i] := true
x:= 1-i
while x != i and b[1-i]:
    skip
// Critical section
b[i] = false
```

Exercise, formulate mutual exclusion and prove it

$$\frac{p_2, m \rightarrow p'_2, m'}{p_1, p_2, m \rightarrow p_1, p'_2, m'}$$

Asynchronous communications

Many forms:

- **Ordered** vs Unordered buffers
- **Unbounded** vs Bounded buffers
- Single vs **Multiple** channels
- **Global** vs Local channels

Single component $(S_i, Act_i, \rightarrow_i, I_i)$ $Act_i \subseteq \{\tau\} \cup C \times \{!, ?\} \times M$

Channel buffers $b : C \rightarrow M^*$

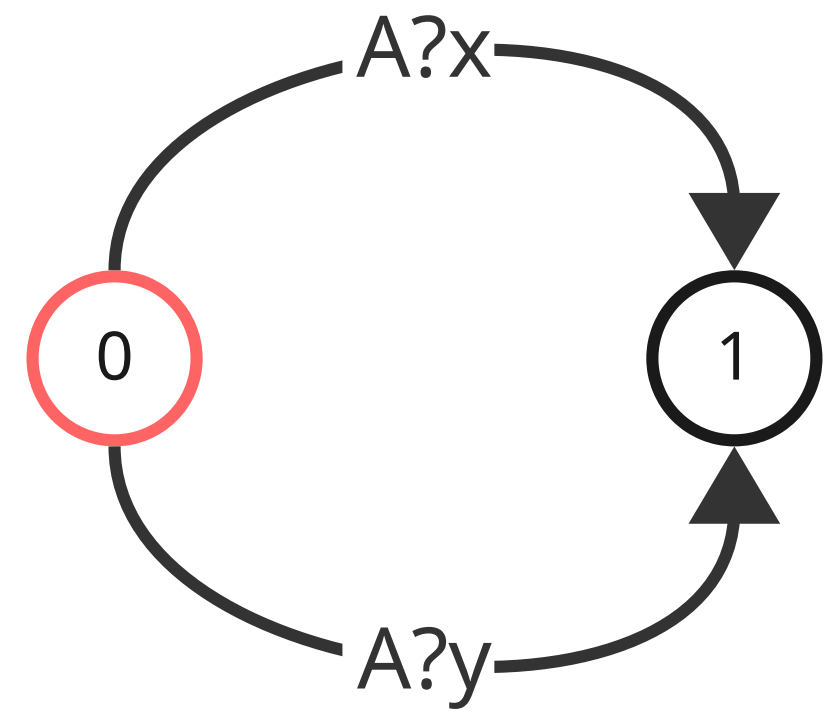
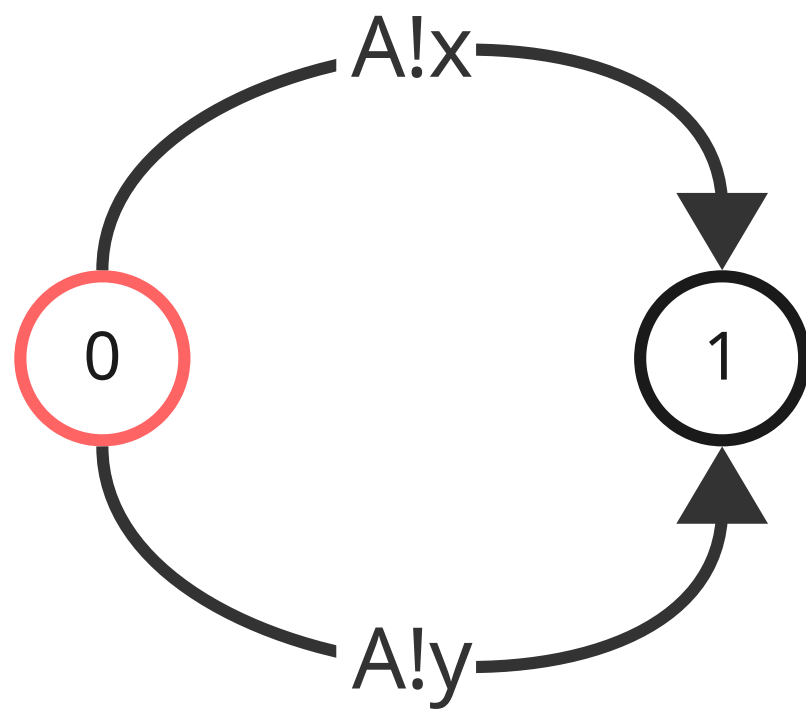
Component states $\sigma = (s_1, \dots, s_n)$

$$\frac{\sigma(i) \xrightarrow{c!m} s' \wedge b(c) = \vec{m}}{\sigma \parallel b \xrightarrow{\tau} \sigma[i \mapsto s'] \parallel b[c \mapsto \vec{m} \cdot m]}$$

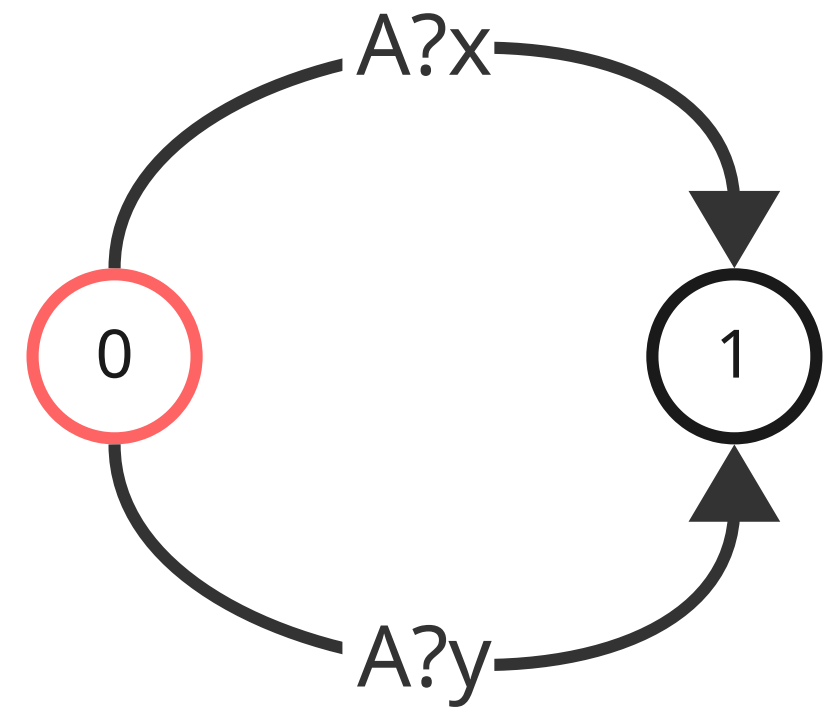
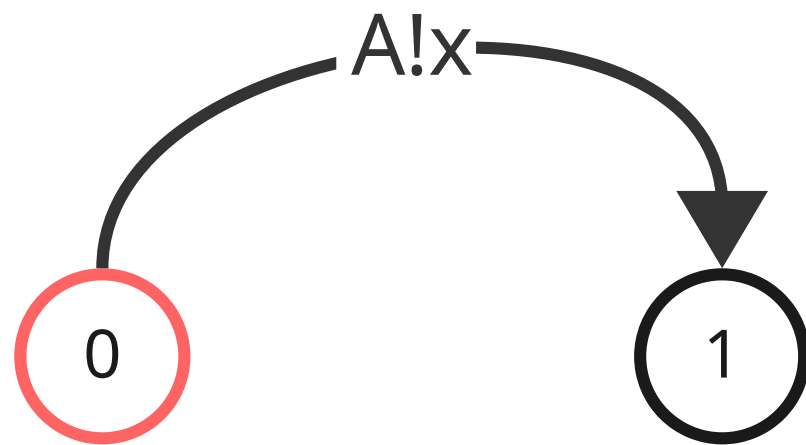
$$\frac{\sigma(i) \xrightarrow{\tau} s'}{\sigma \parallel b \xrightarrow{\tau} \sigma[i \mapsto s'] \parallel b}$$

$$\frac{\sigma(i) \xrightarrow{c?m} s' \wedge b(c) = m \cdot \vec{m}}{\sigma \parallel b \xrightarrow{\tau} \sigma[i \mapsto s'] \parallel b[c \mapsto \vec{m}]}$$

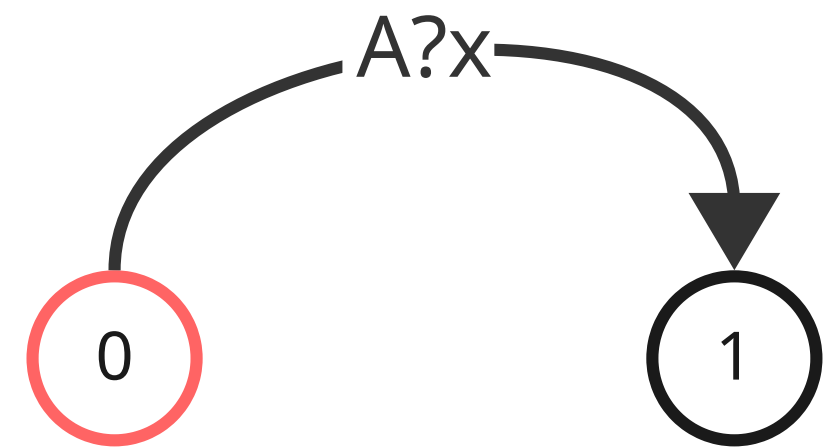
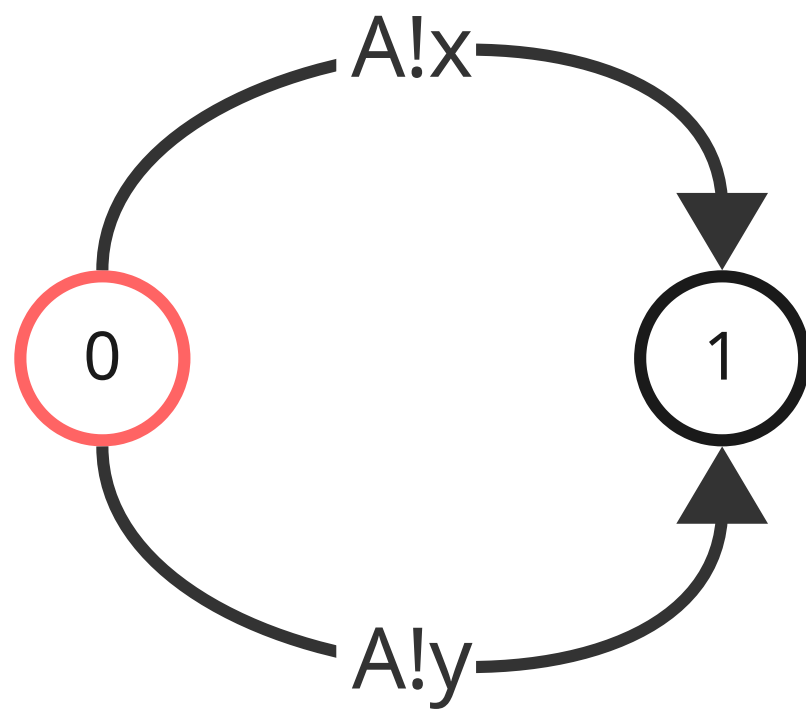
Asynchronous communications



Asynchronous communications

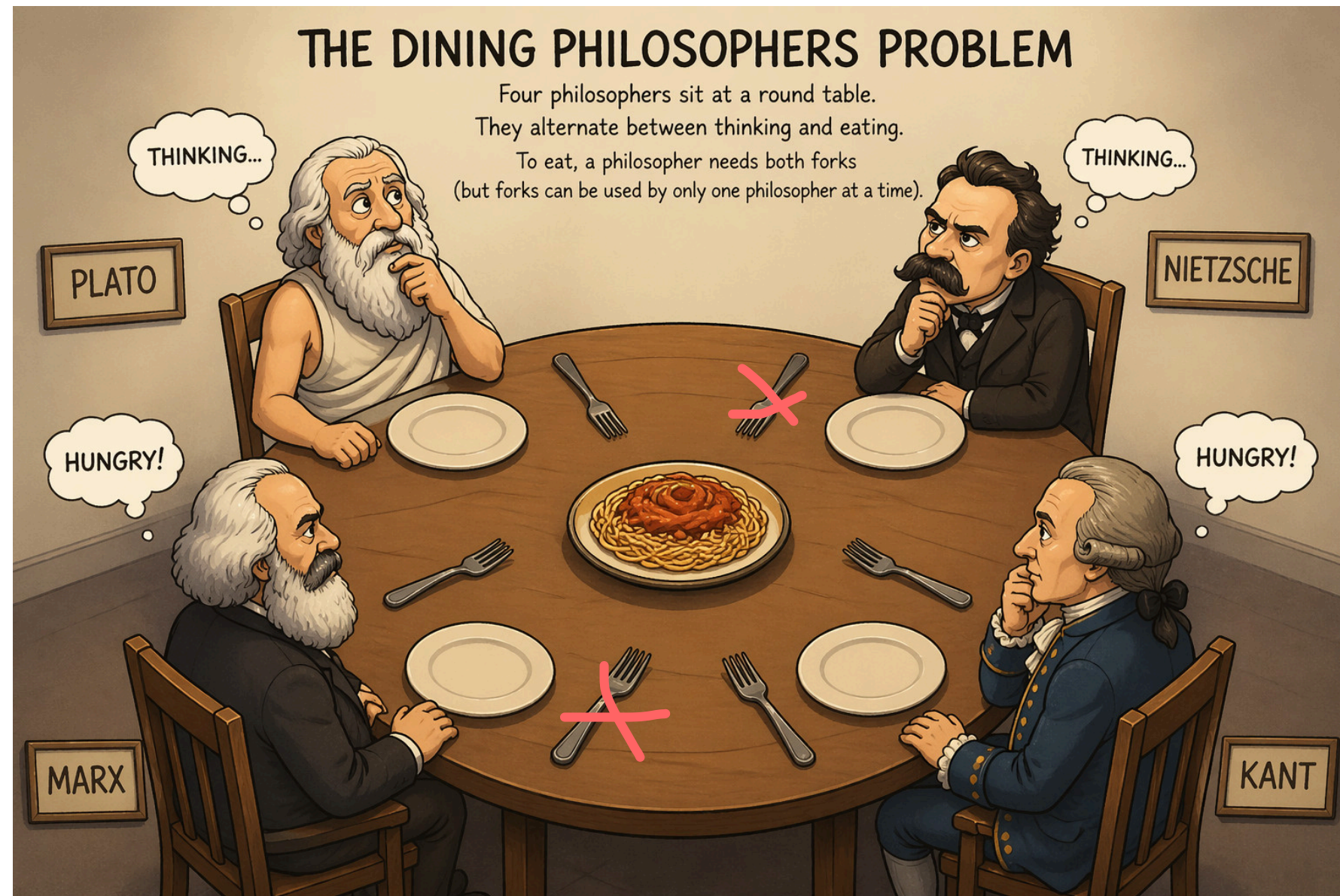


Asynchronous communications

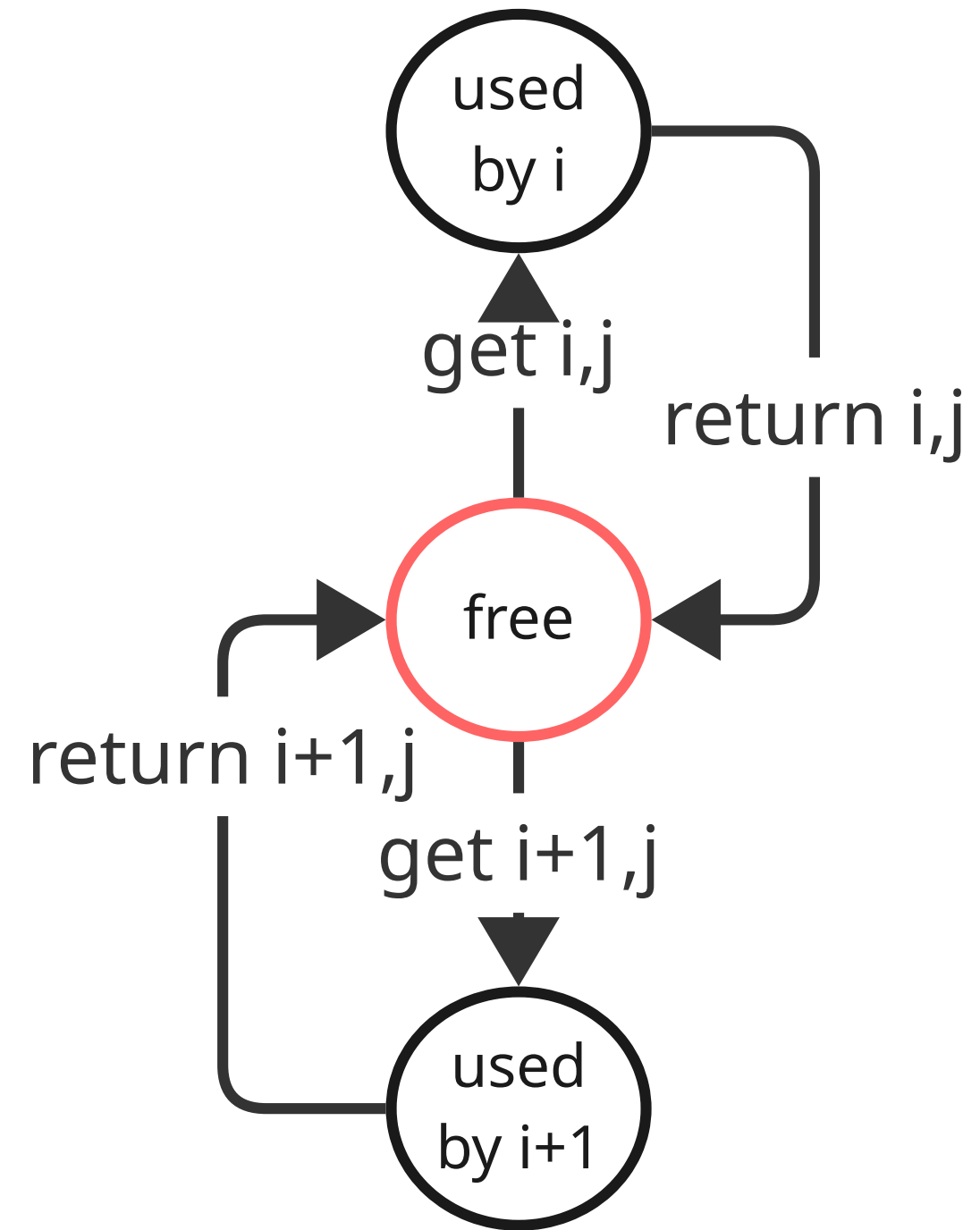
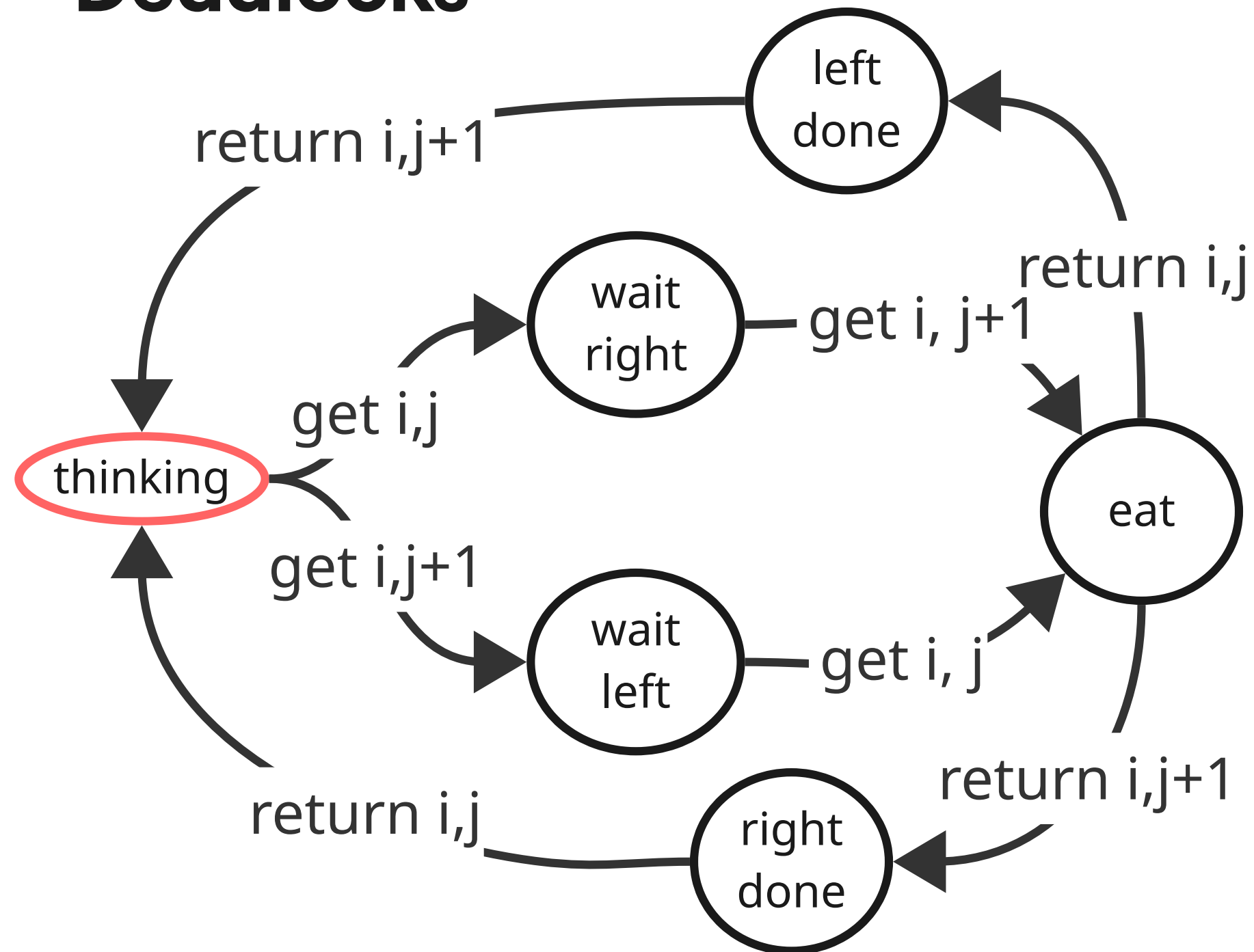


Deadlocks

The dining philosophers



Deadlocks



Remove deadlock, formulate absence of deadlock, prove it

Traces

Let $\rho = s_1\alpha_1s_2\alpha_2 \dots$ be an execution fragment. the trace of the execution fragment is defined as $\pi = \text{trace}(\rho) = \alpha_1\alpha_2 \dots$

Traces can be finite vs infinite, maximal vs partial

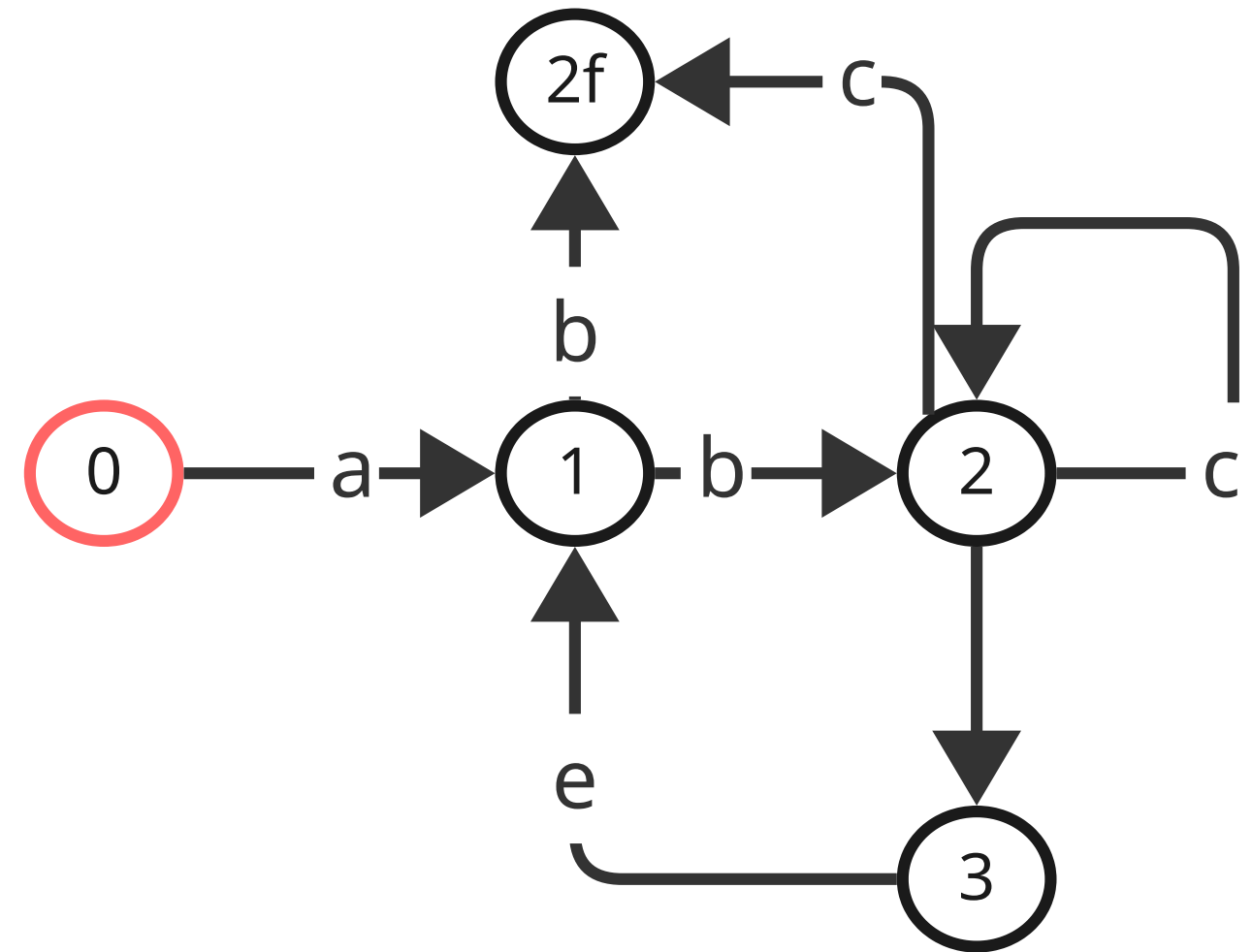
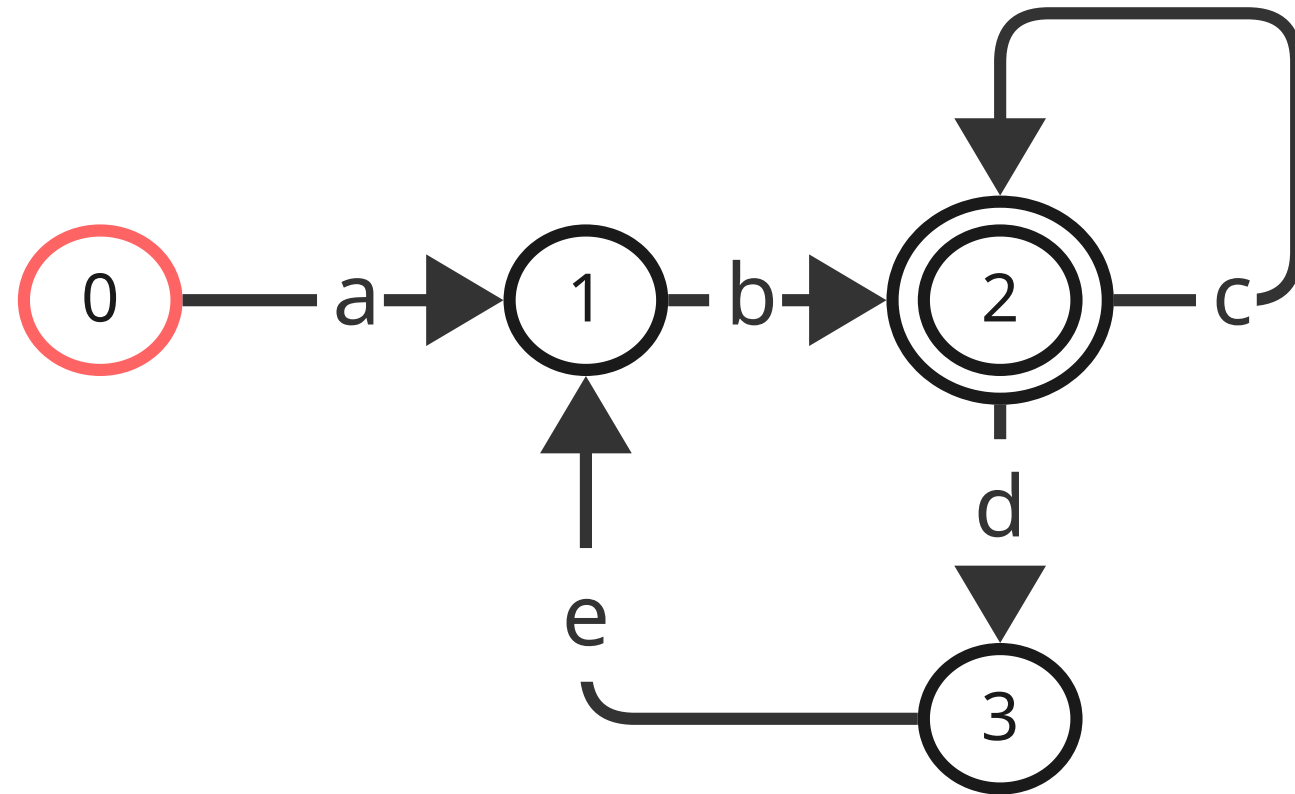
$\text{traces}(TS)$ is the set of traces of initial and maximal path fragments of a transition system

$\text{traces}(s)$ is the set of traces of initial and maximal path fragments of a transition system starting from s

$\text{traces}_{fin}(TS)$ traces of initial and partial path fragments of TS

Can be seen as the Language of the transition system

Relation With DFA



Traces

Let $\rho = s_1\alpha_1s_2\alpha_2 \dots$ be an execution fragment. the trace of the execution fragment is defined as $\pi = \text{trace}(\rho) = \alpha_1\alpha_2 \dots$

Traces can be finite vs infinite, maximal vs partial

$\text{traces}(TS)$ is the set of traces of initial and maximal path fragments of a transition system

NB. In some formalisation there is a labelling function $L(s)$, which returns a set of atomic propositions, and the trace of an execution consists of the labels of the visited states

Hereafter we use Σ for Act or $2^{AtomicProps}$

Linear Time Properties

A linear time property P over a set of labels Act is a subset of Σ^ω

Σ^ω is the set of all possible word that arise from **infinite** concatenations

We restrict now to only transition systems with **no terminal states**

A TS satisfies P iff $traces(TS) \subseteq P$ $TS \models P$

A state satisfies P iff $traces(s) \subseteq P$ $s \models P$

Linear Time Properties

Suppose we extend the Peeterson TS with an outer loop and we use as traces the labeling function that returns the pair of program counters

Mutual exclusion $\{\pi \mid \forall i. (3, 3) \notin \pi[i]\}$

Starvation freedom $\{\pi \mid \forall i. (0, _) \in \pi[i] \Rightarrow \exists j > i \mid (3, _) \in \pi[j]\}$

Trace Inclusion

Two TSs. TS2 is the spec, the TS1 is the implementation

$$\textit{traces}(TS_1) \subseteq \textit{traces}(TS_2)$$

For linear time properties

$$TS_2 \models P \Rightarrow TS_1 \models P$$

Trace equivalence

$$\textit{traces}(TS_1) = \textit{traces}(TS_2)$$

For linear time property this means they satisfy the same properties

Invariants

P_I is an invariant formula iff exists a propositional formula Φ such that

$$P_I = \{\alpha_0\alpha_1\cdots \in \Sigma^\omega \mid \forall j \geq 0. \alpha_j \models \phi\}$$

For example in case of traces being strings of sets of atomic propositions:

$$\alpha_j = \{A, B, C\} \quad \phi = A \vee B \Rightarrow C$$

Safety Properties

P is a safety property iff for every word $\pi \in \Sigma^\omega \setminus P$ there is a finite prefix $\hat{\sigma}$ such that

$$P \cap \{\hat{\pi} \cdot \pi' \mid \pi' \in \Sigma^\omega\} = \emptyset$$

$\hat{\pi}$ is called bad prefix for P

The set of all bad-prefixes is $BadPrefixes(P)$

Safety Properties Example

Mutual exclusion $\{\pi \mid \forall i. (3, 3) \notin \pi[i]\}$

$$\pi' \in \Sigma^\omega \setminus P$$

$$\exists i. (3, 3) \in \pi'$$

$$\hat{\pi}' = \pi'[: i]$$

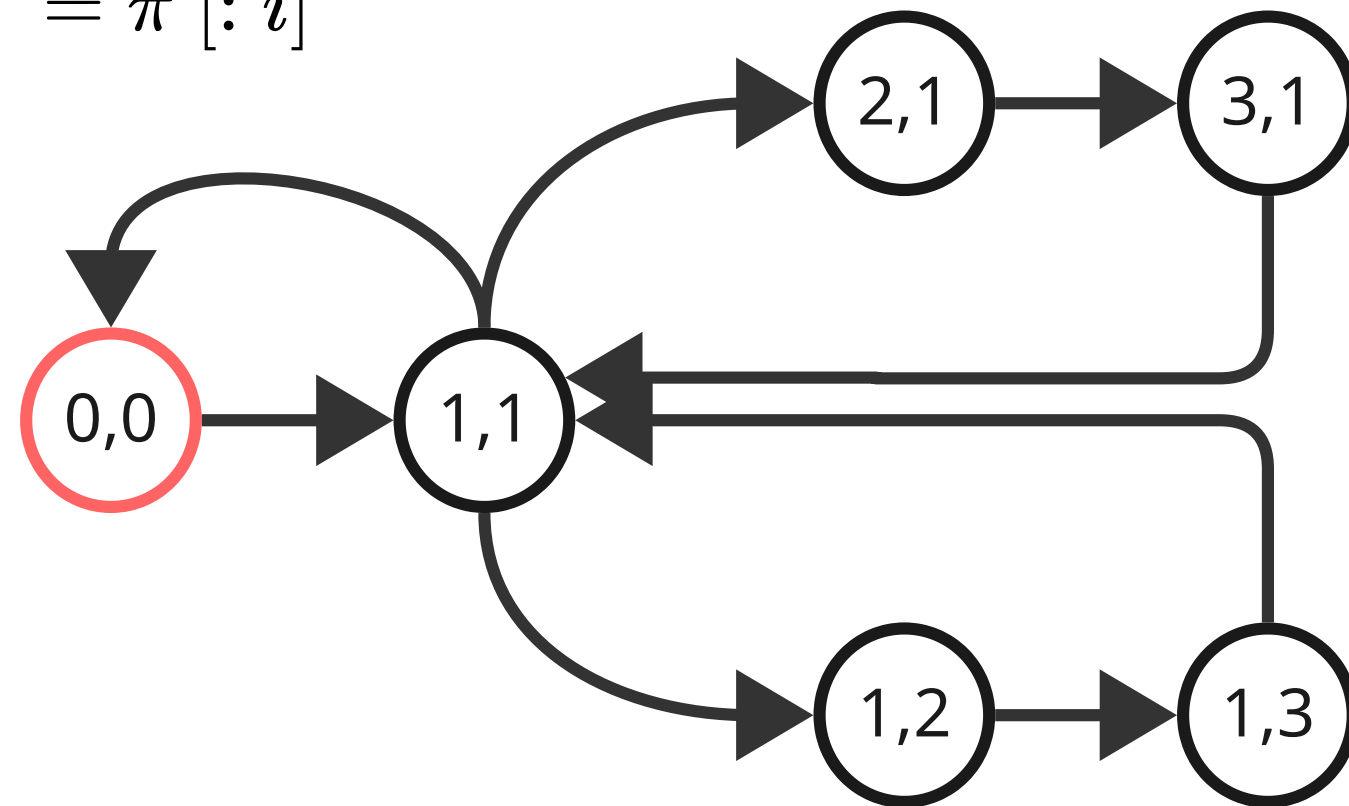
Safety Properties Example

Mutual exclusion $\{\pi \mid \forall i. (3, 3) \notin \pi[i]\}$

$$\pi' \in \Sigma^\omega \setminus P$$

$$\exists i. (3, 3) \in \pi'$$

$$\hat{\pi}' = \pi'[: i]$$



Safety Properties

P is a safety property iff for every word $\pi \in \Sigma^\omega \setminus P$ there is a finite prefix $\hat{\sigma}$ such that

$$P \cap \{\hat{\pi} \cdot \pi' \mid \pi' \in \Sigma^\omega\} = \emptyset$$

$\hat{\pi}$ is called bad prefix for P

The set of all bad-prefixes is $BadPrefixes(P)$

For a safety property:

$$TS \models P \Leftrightarrow traces_{fin}(TS) \cap BadPrefixes(P) = \emptyset$$

$traces_{fin}(TS) \subseteq traces_{fin}(TS') \Leftrightarrow (TS' \models P \Rightarrow TS \models P)$ for every safety property P

Safety Properties

$$TS \models P \Leftrightarrow \text{traces}_{fin}(TS) \cap \text{BadPrefixes}(P) = \emptyset$$

=> by contradiction. Let $TS \models P \wedge \text{traces}_{fin}(TS) \cap \text{BadPrefixes}(P) \neq \emptyset$

$$\exists \hat{\pi} \in \text{traces}_{fin}(TS) \cap \text{BadPrefixes}(P)$$

by def of finite traces, this traces can be extended to

$$\exists \pi = \hat{\pi} \cdot \pi' \in \text{traces}(TS)$$

by definition of safety property

$$\pi \notin P$$

$$TS \not\models P$$

Safety Properties

$$TS \models P \Leftrightarrow \text{traces}_{fin}(TS) \cap \text{BadPrefixes}(P) = \emptyset$$

$\Leftarrow$ by contradiction. Let $TS \not\models P \wedge \text{traces}_{fin}(TS) \cap \text{BadPrefixes}(P) = \emptyset$

$$\exists \pi \in \text{traces}(TS) \wedge \pi \notin P$$

by def of safety property

$$\exists \hat{\pi}. \pi = \hat{\pi} \cdot \pi' \wedge \hat{\pi} \in \text{BadPrefixes}(P)$$

by definition of finite traces

$$\hat{\pi} \in \text{traces}_{fin}(TS)$$

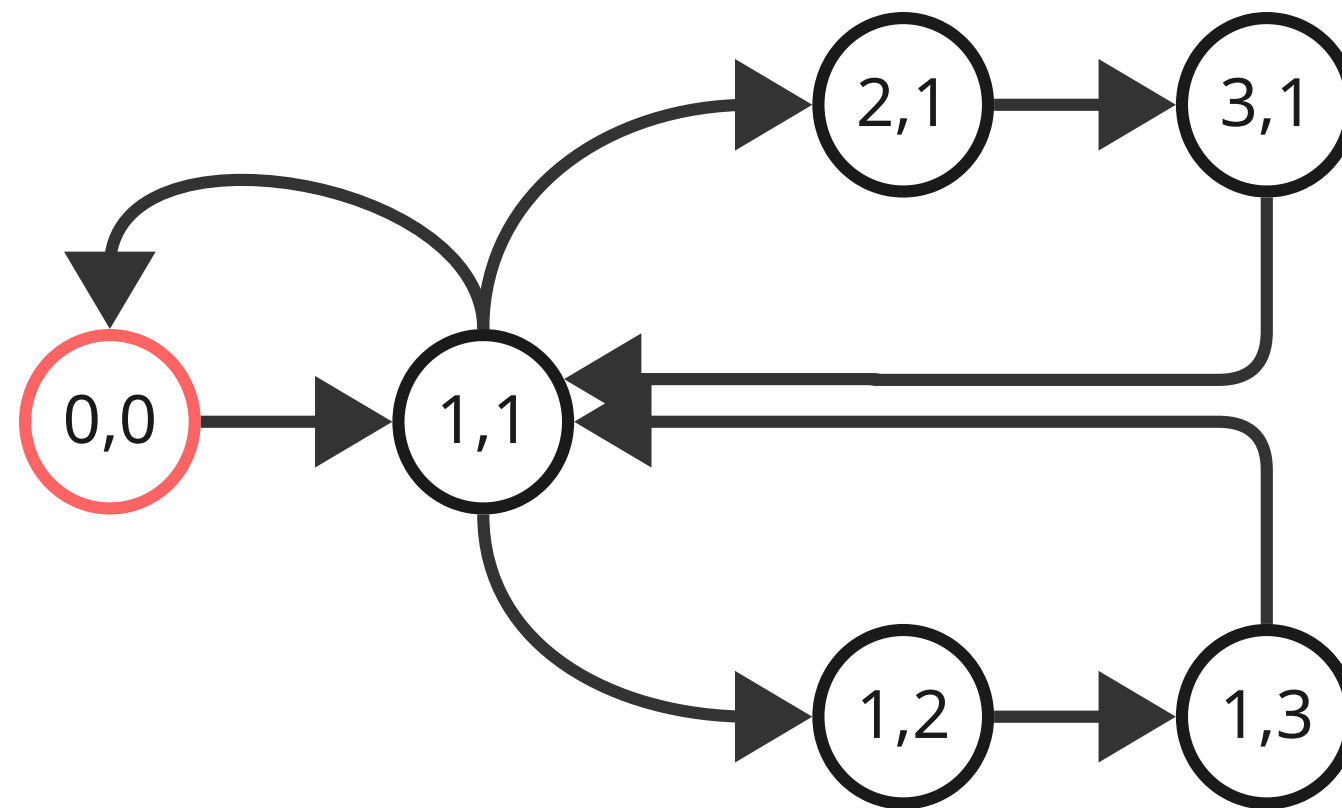
Liveness Properties

P is a liveness property iff $Pref(P) = \Sigma^*$

that is, every prefix can be extended to meet the property

Liveness Properties Example

Starvation freedom $\{\pi \mid \forall i. (0, _) \in \pi[i] \Rightarrow \exists j > i \mid (3, _) \in \pi[j]\}$



Decomposition

The only LT property that is both a safety and liveness property is Σ^ω

For every LT property P there is a liveness P_l and a safety P_s such that $P = P_l \cap P_s$